

2026 年（第十三届）英特尔杯大学生电子设计竞赛嵌入式 AI 专题赛
2026 Intel Cup Undergraduate Electronic Design Contest
- Embedded System Design Invitational Contest

作品设计报告

Final Report



Intel Cup Embedded System Design Contest

报告题目：智驭低空枢纽：面向低空巡检与

火源识别的多模态无人机地面站系统

学生姓名：谢秋实 朱拓源 庞亚宸

指导教师：夏景圆

参赛学校：华中科技大学

2026 年（第十三届）英特尔杯大学生电子设计竞赛

嵌入式 AI 专题赛

参赛作品原创性声明

本人郑重声明：所呈交的参赛作品报告，是本人和队友独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的作品成果，不侵犯任何第三方的知识产权或其他权利。本人完全意识到本声明的法律结果由本人承担。

参赛队员签名： 谢秋实 朱拓源 庞亚宸

日期：2026 年 6 月 30 日

智驭低空枢纽：面向低空巡检与火源识别的多模态无人机地面站系统

摘要

针对低空巡检任务中图传监视、目标识别、飞行控制与事件记录相互分散，导致操作门槛高、误触发风险大和任务过程难以追溯的问题，本文设计并实现了一套面向低空巡检与火源识别的多模态无人机地面站系统。系统以 Intel DK-2500 地面端为智能枢纽，接入 AMB82 实时图传，结合镜头校正、火源检测、连续帧确认、任务地图、飞控数传接口和事件日志，实现“目标发现—候选生成—人工确认—飞控执行—状态回传—任务记录”的闭环流程。在交互层面，系统融合语音识别、手势识别、视线选择和面板按钮等多种输入方式，将起飞、自检、巡线、火源确认、返航和降落等复杂飞控操作封装为任务级命令，并通过安全命令中心对高风险动作进行二次确认。测试结果表明，系统能够稳定完成图传接入、火源候选识别、多模态指令生成、巡线任务执行、状态回传和事件留痕，具备较好的低成本部署、低培训使用和安全可追溯特点，可为教学竞赛、原型验证和轻量化低空巡检场景提供参考。

关键词：无人机地面站；低空巡检；火源识别；多模态交互；人机协同；安全可追溯闭环

LOW-ALTITUDE INTELLIGENT HUB: A MULTIMODAL UAV GROUND STATION FOR PATROL AND FIRE DETECTION

ABSTRACT

To address the problems of fragmented video monitoring, target recognition, flight control, and event recording in low-altitude patrol tasks, this report presents a multimodal UAV ground station system for low-altitude patrol and fire-source detection. The system uses the Intel DK-2500 platform as the ground-side intelligent hub and integrates AMB82 real-time video streaming, lens correction, fire-source detection, multi-frame confirmation, mission map visualization, flight-control data transmission, and event logging. Through this design, a complete workflow is established, covering target detection, candidate generation, human confirmation, flight execution, telemetry feedback, and task recording.

At the interaction level, the system combines speech recognition, gesture recognition, gaze-based selection, and panel buttons to transform complex flight operations, such as takeoff, self-check, patrol, fire-source confirmation, return-to-launch, and landing, into task-level commands. To reduce the risk of false triggering, all high-risk commands are first generated as candidate commands and then verified through a safety command center before execution. Test results show that the system can stably complete video stream access, fire-source candidate detection, multimodal command generation, patrol task execution, status feedback, and traceable event recording. The proposed system features low-cost deployment, low training requirements, and a safety-oriented human-in-the-loop workflow, making it suitable for teaching competitions, prototype verification, and lightweight low-altitude patrol scenarios.

Key words: UAV ground station; low-altitude patrol; fire-source detection; multimodal interaction; human-in-the-loop control; safety traceability loop.

目 录

第一章 绪论.....	1
1.1 项目背景与应用场景.....	1
1.2 竞赛任务理解与作品定位.....	1
1.3 现有无人机地面站与低空巡检系统存在的问题.....	1
1.4 本作品的总体思路与创新点概述.....	2
第二章 系统总体方案.....	4
2.1 系统需求分析.....	4
2.2 总体架构设计.....	5
2.3 硬件组成与通信链路.....	5
2.4 软件总体架构.....	8
2.5 系统工作流程与安全闭环.....	9
第三章 功能介绍.....	10
3.1 功能一：语音识别启动.....	10
3.2 功能二：手势识别与视线选项组合.....	11
3.3 功能三：面板按钮启动.....	13
3.4 功能四：起飞、自检与返航专项流程.....	13
3.5 功能五：巡线与火源识别.....	14
3.6 功能六：重要事项鼠标二次确认.....	16
3.7 功能七：地面站 UI 模块说明.....	17
第四章 飞机平台、任务地图与巡逻路线设计.....	18
4.1 无人机平台组成.....	18
4.2 机载摄像模块与地面站图传关系.....	19
4.3 起飞与降落的关键控制逻辑.....	20
4.4 任务地图与路线规划实现.....	20
4.5 航点执行状态机与火源识别联动.....	22
4.6 数传工作原理与飞控—地面站联动流程.....	23
第五章 地面站与飞控闭环关键技术实现.....	25
5.1 地面站主窗口与任务总线.....	25
5.2 AMB82 图传接入、镜头校正与火源检测.....	25
5.3 连续帧确认与候选目标生成.....	26
5.4 多模态候选指令生成机制.....	26
5.5 安全命令中心与二次确认状态机.....	27
5.6 数传任务接口与飞控状态回传.....	27
5.7 任务地图、事件日志与可追溯记录.....	28
5.8 完整闭环工作流实现.....	28
第六章 系统闭环测试与结果分析.....	29
6.1 测试环境与评价指标.....	29
6.2 图传链路与界面响应测试.....	29
6.3 火源识别与连续确认测试.....	29
6.4 多模态候选命令测试.....	30
6.5 安全命令与飞控任务闭环测试.....	30
6.6 地图巡线与状态回传测试.....	31
6.7 测试结果汇总与系统应用价值.....	31
6.8 稳定性分析与扩展方向.....	32
第七章 总结与展望.....	32

参考文献.....	33
附录 A 主要程序文件说明.....	34

第一章 绪论

1.1 项目背景与应用场景

低空无人机已广泛应用于园区巡检、灾害预警、消防辅助、设备巡查和教学竞赛等场景。在这些任务中，无人机不仅需要稳定完成航线飞行，还需要把实时图像、飞行状态、目标事件和操作者命令集中呈现给地面端。传统遥控方式更强调飞行控制本身，而对任务过程中的信息组织、目标确认和事件留痕支持不足。

火源巡检是低空无人机应用中具有代表性的任务。操作者需要在飞行过程中同时观察图传画面、判断目标状态、关注电池和链路安全，并在必要时执行暂停、返航或降落等命令。如果缺少清晰的地面站界面，目标识别结果很容易与飞行控制流程脱节，影响任务效率与安全性。

本作品面向低空巡检与火源识别场景，构建“智驭低空枢纽”多模态无人机地面站系统。系统以实时图传为核心，以火源检测、语音交互、任务地图和安全命令中心为支撑，使地面站从单纯监视界面提升为任务组织与人机协同枢纽。

1.2 竞赛任务理解与作品定位

英特尔杯嵌入式 AI 专题赛强调基于英特尔处理器平台的嵌入式系统设计、AI 能力展示和完整作品实现。本作品并不把目标局限于单一算法验证，而是围绕无人机巡检这一完整应用，把图像输入、状态显示、智能识别、任务控制和多模态交互组织成可演示的系统。

作品定位为“面向低空巡检的多模态无人机地面站”。其中，地面站承担系统集成与交互展示任务；AMB82 摄像模块承担实时图像输入；火源检测模块承担目标候选生成；语音、手势和视线输入承担多模态交互扩展；飞控程序与任务地图共同承担巡逻路线和任务流程表达。

1.3 现有无人机地面站与低空巡检系统存在的问题

成熟无人机地面站通常具备航线规划、地图显示和遥测监视能力，但在竞赛原型和小型应用中，常见问题包括界面信息层级混乱、图传与目标识别分离、多模态输入缺少统一安全流程、事件日志不完整、飞控任务路线与地面端展示脱节等。

在火源识别任务中，单纯显示一个检测框并不能构成完整作品。评审更关注系统是否能够解释目标如何产生、操作者如何确认、命令如何下发、事件如何记录，以及出现误识别或高风险命令时系统如何避免不安全执行。因此，地面站必须具备明确的人机闭环和可追溯机制。

图 1-1 从信息组织、目标确认、安全命令、多模态输入和事件追踪五个方面概括了低空巡检系统的典型痛点。本作品以统一地面站为核心，将图传、识别、确认、控制和记录纳入同一闭环，以提高任务执行的可解释性和安全性。

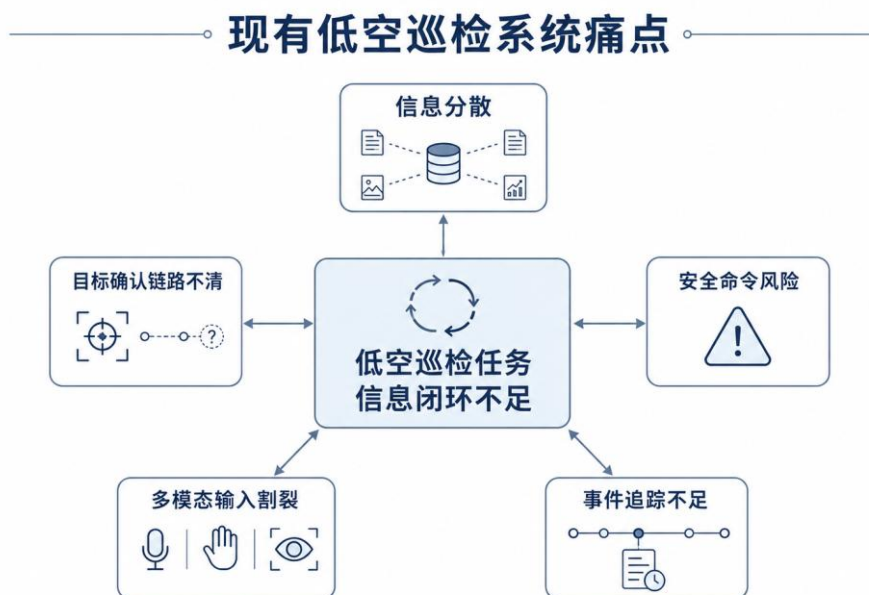


图 1-1 现有低空巡检系统痛点分析图

1.4 本作品的总体思路与创新点概述

本作品总体思路是构建“感知—候选—确认—执行—记录”的低空巡检闭环。实时图传和飞行状态构成基础感知；火源检测、多模态输入和任务地图事件构成候选信息；安全命令中心承担人工确认与风险控制；飞控任务状态机完成动作执行；事件日志和系统日志实现全过程记录。与单纯展示语音、手势或识别算法不同，本作品更强调低成本、低培训、低步骤与高安全的系统级创新，使地面站成为连接飞行端、操作者和任务数据的低空智能枢纽。

第一，低成本模块化飞行端。本作品面向教学竞赛、低空巡检原型验证和轻量化火源辅助巡查场景，采用低成本模块化飞行端与地面智能枢纽组合，避免直接依赖高价一体化救援无人机。系统把图传接入、火源识别、任务确认和日志记录等智能能力集中在地面站侧，使飞行端保持低成本、易替换、易维护的结构。需要强调的是，本作品并不定位为替代工业级救援无人机，而是提供一套适合教学竞赛、原型验证和轻量巡查的低成本任务流程平台。**表 1-1 低成本飞行端与商业行业无人机方案对比**

方案	主要组成	估算成本	特点
本作品低成本飞行端	AMB82 图传、简易飞控、无人机骨架、4 电机、电调、电池、数传模块	约 ¥1500—¥3500	成本低、可拆换，适合教学竞赛和原型验证
商业救援/行业无人机参考	DJI Matrice 30T / M30T 套装，集成广角、变焦、热成像、激光测距等	约 ¥8—10 万级	工业可靠性强，但采购和维护成本高
本作品地面智能枢纽	Intel DK-2500 + 地面站软件 + 多模态输入 + AI 识别	竞赛平台复用	将智能能力集中到地面端，降低飞行端复杂度

注：AMB82-Mini 公开零售价格约 29.99 美元；DJI Matrice 30T 公开零售页面常见价格约 12602—14452 美元，按 2026 年美元兑人民币约 6.8—6.9 的汇率粗略折算，可作为 8 万元人民币以上级别的参考区间。相关价格为 2026 年 6 月公开渠道检索结果，仅用于竞赛报告中的方案量级对比，实际采购价格会随渠道、套装置置和汇率变化。本方案定位为教学竞赛与轻量级巡检

原型，不直接替代工业级救援无人机。上述价格不代表同等性能对比或采购建议^[7-10]。

第二，低培训门槛的人机交互。系统通过语音、手势和视线三卡片将复杂飞控动作封装为任务级操作，使操作者不需要直接记忆大量遥控器杆量、飞控模式和航线设置步骤，而是通过“选择任务—确认执行—观察反馈”的方式完成巡检流程。该设计降低了非专业人员的入门门槛，也使应急场景下的操作更接近自然交互。

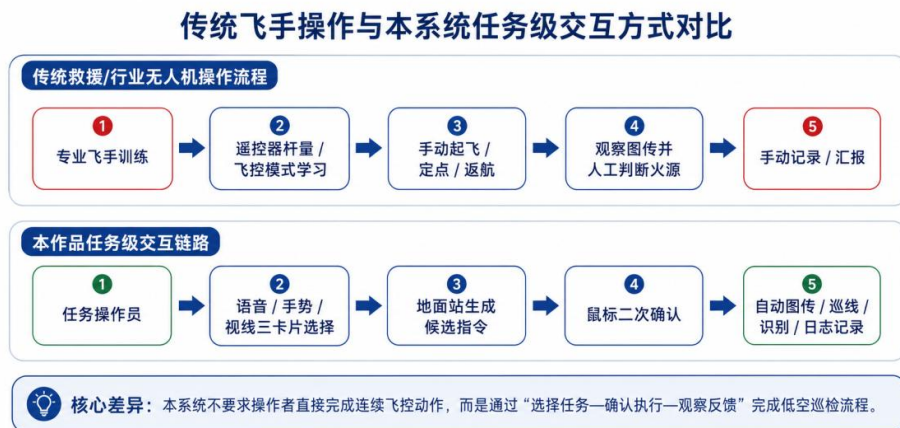


图 1-2 传统飞手操作与本系统任务级交互方式对比

第三，预封装任务命令与快速响应。系统将预热自检、低空起飞、开始巡线、火源识别、悬停、返航和降落等任务动作封装为标准流程。在危险或紧急场景中，操作者面对的不是大量分散控件和连续飞控操作，而是清晰的候选任务、风险等级和确认按钮，从而降低认知负担并减少漏操作、误操作。

表 1-2 预封装任务命令与操作步骤对比

任务类型	传统操作流程	本系统操作流程	减少步骤	安全机制
起飞前检查	人工检查飞控、电机、摄像头、链路等多个环节	点击或选择“预热自检”，系统生成状态报告	多项人工检查压缩为一次任务入口	异常状态进入日志并持续提示
低空起飞	切换飞控模式并连续操作遥控器或多个控件	选择“低空起飞”并完成二次确认	连续杆量控制转化为任务级命令	起飞属于高风险动作，需确认后执行
巡线任务	人工控制航向或逐项设置航点	选择“开始巡线”，地面站下发预设航点序列	减少航点配置与现场操作步骤	任务进度与航点状态实时回传
火源确认	人眼观察、手动截图、手动记录位置	识别框、事件中心和地图红点同步提示	发现、标记和记录合并为统一流程	候选目标经人工确认后写入事件记录
返航/降落	切换模式并依赖飞手判断执行	选择返航或降落，安全命令中心统一确认	把应急处置简化为明确按钮流程	高风险命令二次确认并记录回执
异常处理	依赖飞手经验观察链路、电量和画面异常	状态栏、设备健康区和系统日志持续提示	减少分散观察和漏判风险	异常事件可复盘、可追踪

第四，安全可追溯闭环。语音、手势、视线和按钮输入均不直接控制无人机，而是先生成候选命令或候选目标，再由安全命令中心进行风险分级和人工确认。所有高风险命令均经过候选生成、人工确认、执行反馈和日志记录，形成可复盘、可追踪的任务链路。由此，本作品的多模态能力不是单纯的交互展示，而是服务于“低成本飞行端 + 地面智能枢纽 + 人工安全确认”的系统创新。

第二章 系统总体方案

2.1 系统需求分析

结合低空巡检任务，本系统的需求可分为功能需求、性能需求和安全需求。功能上需要完成实时图传、火源识别、状态监视、任务地图、多模态交互和命令管理；性能上需要保证图传刷新率、界面响应和识别时延满足演示要求；安全上需要对高风险命令进行确认，并对关键事件进行记录。

表 2-1 系统需求与设计响应

需求类别	具体需求	设计响应
图传监视	实时显示机载画面并保持画面比例	AMB82 RTSP 图传、16:9 视频画布、快照回退
目标识别	识别可见火源或模拟火源标志	火源检测算法、连续帧确认、检测框叠加
状态感知	展示高度、电量、链路、任务进度	右侧状态栏、设备健康面板、任务地图
多模态交互	支持语音、手势、视线输入	多模态交互 Dock 与候选命令机制
安全控制	避免误触发高风险飞行动作	安全命令中心、二次确认、事件日志

2.2 总体架构设计

系统总体架构由机载侧、通信链路和地面站侧组成。机载侧包含无人机平台、飞控程序和 AMB82 摄像模块；通信链路包含 RTSP 图传、串口或遥测链路以及命令下发链路；地面站侧包含 PyQt5 主界面、图传线程、火源识别、多模态交互、任务地图、事件日志和安全命令中心。

图 2-1 展示了多模态输入、Intel 板卡地面站、外设设备与无人机飞控系统之间的关系。AMB82 与地面站通过无线局域网传输 RTSP 图传，无人机与地面站通过数传链路传输遥测数据和任务命令，形成面向低空巡检的软硬件协同架构。

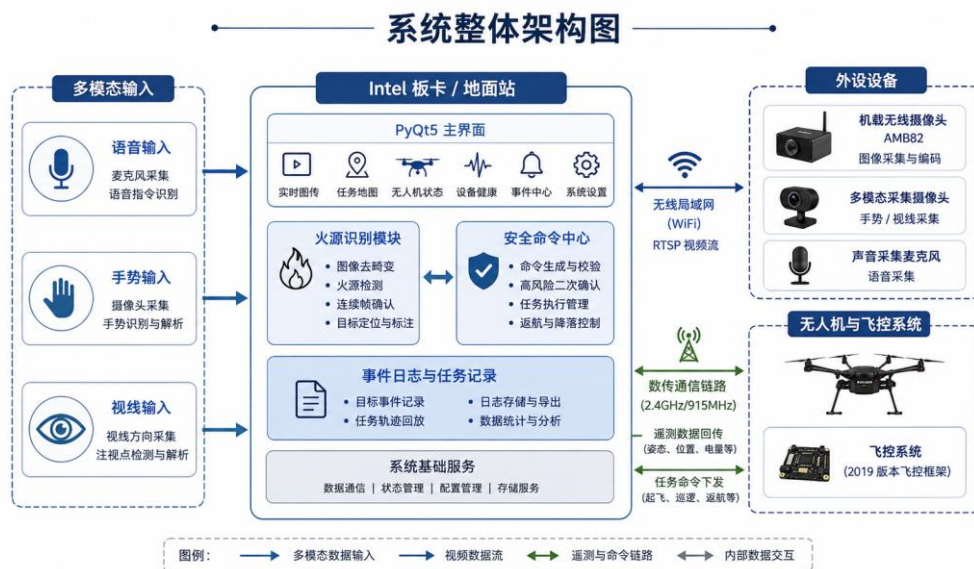


图 2-1 系统整体架构图

2.3 硬件组成与通信链路

硬件层面，系统由板卡端与无人机端两部分组成。板卡端以 Intel DK-2500 为核心，承担地面站主界面显示、图传接入、火源识别、多模态交互与任务调度，并配套显示器、麦克风、多模态采集摄像头、鼠标键盘和 2.4G/5.8G 网络接口等外设；无人机端以 F260-T432 四旋翼平台为载体，集成飞控开发板、AMB82-mini 摄像头图传、nRF24L01 数传模块、电池、电调与电机，用于低空巡逻、图像采集和遥测回传。为使系统组成更加清晰，表 2-2 给出了板卡端和无人机端的总体硬件清单，表 2-3 进一步概括关键硬件之间的通信关系。



图 2-2 板卡端 / 无人机端实物对应图

图 2-2 将 Intel DK-2500 板卡端、地面站软件界面、AMB82-mini 图传模块、数传模块、飞控开发板、动力系统与 F260-T432 无人机平台进行对应展示，用于说明表 2-2 和表 2-3 中各硬件模块在地面端与无人机端的实际部署位置及数据交互关系。

表 2-2 系统总体硬件清单

端别	类别	硬件/模块	主要作用
板卡端	计算与控制核心	Intel DK-2500 开发板	运行地面站程序，完成图传接入、火源识别、多模态交互、任务地图与命令调度。
板卡端	显示与交互	外接显示器	显示实时图传、状态面板、任务地图、事件日志和命令中心。
板卡端	多模态输入	USB 麦克风	采集语音输入，供 OpenVINO Whisper 实时识别。
板卡端	多模态输入	多模态采集摄像头	采集手势与视线信息，用于手势识别与三选一视线交互。
板卡端	人机操作	鼠标与键盘	执行目标确认、参数设置和安全命令确认操作。
板卡端	网络与连接	2.4G/5.8G 网卡 / 路由网络	与 AMB82 建立无线局域网连接，接收 RTSP 视频流。
无人机端	飞行平台	F260-T432 四旋翼平台	作为低空任务执行节点，完成起飞、巡逻、悬停、返航与降落。
无人机端	飞控核心	F260-T432 飞控开发板	完成姿态解算、PID 控制、电机混控、航点执行与状态回传。
无人机端	图传感知	AMB82-mini 摄像头模块	采集任务画面并通过 RTSP 向地面站回传视频，用于实时显示。
无人机端	数传通信	nRF24L01 / UART 数传模块	在飞控与地面站之间传输遥测状态、任务命令和 ACK 回执。
无人机端	动力系统	无刷电机 + 电调 ESC	输出推力，实现姿态调节。
无人机端	供电系统	LiPo 电池与供电模块	为飞控、图传和动力系统供电。

表 2-3 硬件组成与通信关系

组成部分	主要作用	通信或数据形式
无人机平台	承载飞控、摄像模块和电源系统	机载任务执行
AMB82 图传模块	采集并发送实时视频	RTSP 图传或快照图像
地面站主机	运行界面、识别与交互逻辑	PyQt5、OpenCV、OpenVINO
飞控程序	执行起飞、巡逻、返航、降落流程	串口遥测、命令帧
操作者输入设备	提供语音、手势、视线等交互信号	麦克风、本地摄像头、鼠标键盘

2.4 软件总体架构

软件总体架构采用组件化设计。主窗口负责布局和业务调度；视频服务线程负责相机连接、图像读取和状态汇报；火源识别模块负责候选目标生成；多模态交互模块负责语音、手势与视线输入；数据模型层统一管理无人机状态、相机状态、检测事件和候选命令。



图 2-3 软件总体架构与任务流程示意图

图 2-3 以图像主导的形式展示系统从多模态采集、地面站感知、候选确认到任务执行与记录的完整流程。该流程强调视觉、语音、手势和视线等信息首先进入感知与候选环节，再由安全命令中心完成确认，最终通过飞控执行与事件日志实现闭环记录。

2.5 系统工作流程与安全闭环

系统工作流程结合第三章的功能闭环，可概括为“多源输入—候选生成—安全确认—任务执行—结果记录”五个阶段。语音识别、手势/视线交互以及面板按钮首先产生候选命令，实时图传与火源识别产生候选目标；随后地面站在安全命令中心中统一展示候选信息，并按照风险等级决定是否需要进行二次确认。对于起飞、返航、降落、开始巡逻等高风险动作，系统要求操作者显式确认后才下发到飞控；对于检测告警、截图与普通提示等低风险事件，系统可直接记录并更新界面状态。执行完成后，任务地图、设备状态栏、最近事件与系统日志同步刷新，从而形成可追溯的任务闭环。

传统飞手操作与本系统任务级交互方式对比



图 2-4 系统工作流程与安全闭环示意图

图 2-4 结合第三章功能设计，进一步体现了本作品的核心思想：多模态输入和火源识别模块只负责提出候选目标或候选命令，最终执行权保留在统一的安全命令中心。地面站在确认后驱动任务执行，并同步更新任务地图、状态面板和事件日志，从而兼顾任务效率、系统安全性与全过程可追溯性。

第三章 功能介绍

本章按实际演示流程重新组织功能说明，不再单纯按照界面模块罗列。地面站将“语音、手势+视线、面板按钮”作为三类任务启动入口，将起飞、自检、返航、巡线与火源识别作为主要任务闭环，并在所有高风险动作前加入候选指令与二次确认机制。

系统的设计原则是：多模态输入只产生候选意图，地面站负责统一显示、校验与执行；摄像头图传作为主工作区，状态、地图、设备链路与安全按钮保持常驻可见。

多模态交互的设计重点并非“炫技式输入”，而是让不同环境下的操作者拥有合适入口：语音适合双手不便时快速发起任务；手势适合嘈杂环境下进行简单选择；视线或头部方向适合在三张候选卡片之间快速定位；面板按钮则作为调试和应急兜底入口。所有入口最终汇入同一套候选指令、安全确认和日志记录机制，保证交互自然性与飞行安全性同时成立。

3.1 功能一：语音识别启动

语音识别用于在操作员双手不便或需要快速发起任务时生成候选指令。使用时先在左侧“多模态交互”面板勾选“启用语音输入”，点击“启动实时识别”后，系统持续接收麦克风语音并调用实时转写模块。识别结果不会直接控制无人机，而是进入安全命令中心，等待人工确认或自动归类。

语音识别成功后，界面会显示原始转写文本、候选动作、置信度、风险等级与是否需要确认。高风险指令如起飞、返航、降落、开始巡逻必须经鼠标二次确认后才会执行。



图 3-1 语音识别成功并生成候选指令示意图

表 3-1 语音识别的命令集

语音关键词	候选动作	风险等级	执行策略
开始巡逻 / 开始任务	START_MISSION	高	进入安全命令中心，需确认
起飞 / 自动起飞	TAKEOFF	高	需确认后进入起飞流程
返航 / 返回起点	RTL	高	需确认后进入返航流程
降落	LAND	高	需确认后执行降落
暂停 / 悬停	HOLD	中	生成暂停悬停动作
继续	CONTINUE	中	恢复当前任务
截图	SNAPSHOT	低	保存当前图传画面

3.2 功能二：手势识别与视线选项组合

手势与视线用于构成“两阶段、三卡片”的快速选择流程。第一阶段识别手势，确定任务大类；第二阶段在主界面显示三张候选卡片，由操作员通过头部/视线方向选择左、中、右选项。该流程避免了“识别到一个手势就直接发命令”的误触问题。

目前手势识别保留三种最稳定、最容易展示的动作：张开手掌、握拳、竖大拇指。视线识别采用头部方向优先、眼动辅助的三点选择方式，可在左、中、右三个区域中选择细项。



图 3-2 手势识别示例：张开手掌和握拳

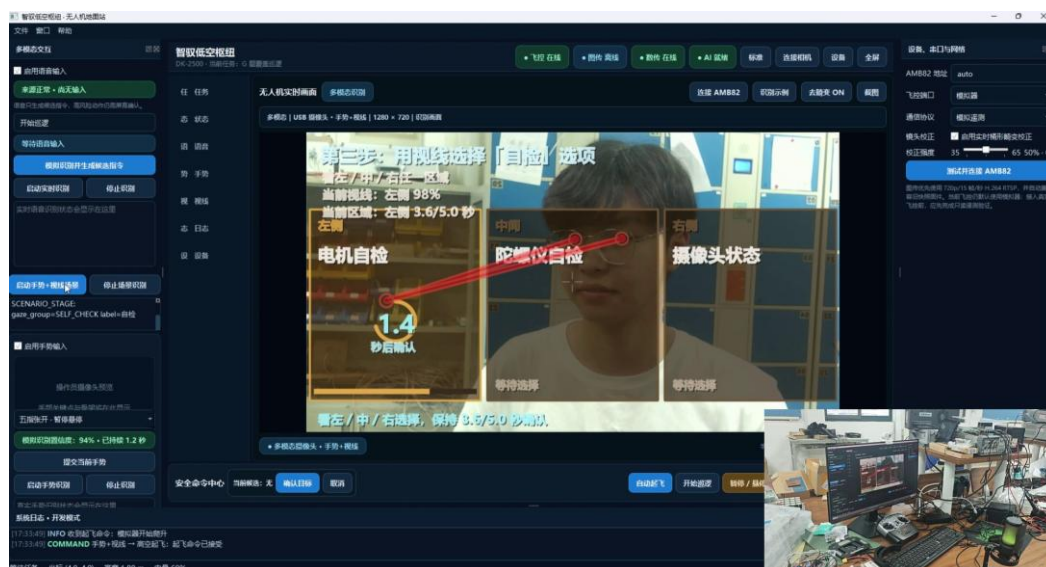


图 3-3 手势确定大类后，视线在三张卡片中选择细项

表 3-2 手势识别的 9 种任务列举

第一阶段手势	任务大类	左侧视线	中间视线	右侧视线
张开手掌 Open Palm	起飞	预热	低空起飞	高空起飞
握拳 Fist	自检	电机自检	陀螺仪自检	摄像头状态检查
竖大拇指 Thumb Up	返航	立即返航	1 分钟后返航	3 分钟后返航

具体操作流程为：点击“启动手势+视线场景”后，系统先进入手势识别阶段；当某个手势稳定保持一段时间后，地面站切换到对应任务组，并在主画面中显示三张卡片；随后通过头部/视线方向选择具体卡片，候选结果进入安全命令中心；若属于高风险动作，则继续等待鼠标确认。

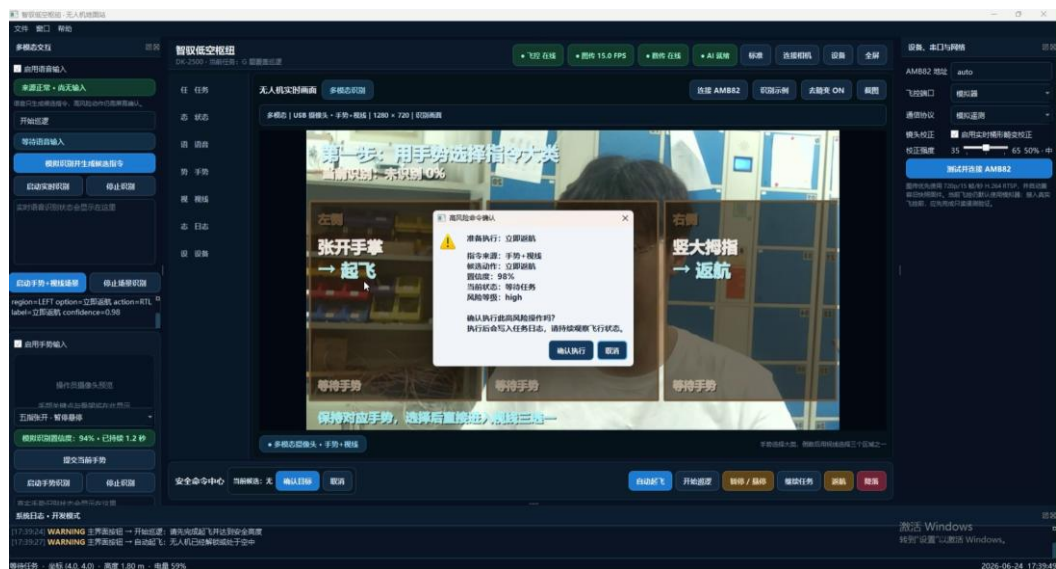


图 3-4 高风险候选动作进入二次确认流程

3.3 功能三：面板按钮启动

除语音、手势与视线外，地面站仍保留传统面板按钮作为第三种启动方法。该方式适合调试、比赛前检查和紧急接管。面板按钮包括自动起飞、开始巡逻、暂停/悬停、继续任务、返航、降落、确认目标、取消等功能。

面板启动的优势是确定性强、可直接复现；多模态启动的优势是展示效果明显、交互更自然。两类入口最终都会汇入同一套候选指令、安全确认、任务状态更新和事件日志机制。

3.4 功能四：起飞、自检与返航专项流程

3.4.1 一键起飞：预热、低空起飞与高空起飞

起飞流程分为预热、低空起飞和高空起飞三种细项。预热不直接进入飞行画面，而是输出飞行器预热报告，用于展示陀螺仪、电机和摄像头链路已经准备完毕；低空起飞用于安全演示，主要展示低高度、低风险的起飞画面；高空起飞保留为更高目标高度的展示入口，用于说明系统可扩展到不同任务高度。

候选动作确认后主画面会先出现“正在切换图传/加载摄像头”的切换动画，然后进入相应摄像头或视频画面。

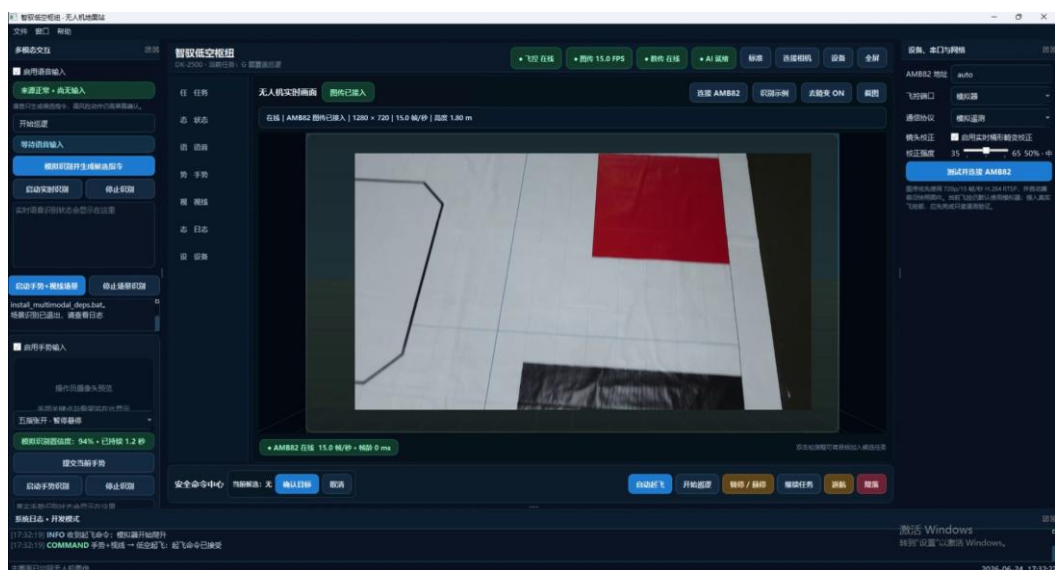


图 3-5 一键起飞流程中的低空飞行画面

3.4.2 系统自检

系统自检用于在任务开始前检查关键设备状态。当前演示版将自检报告分为陀螺仪/IMU、电机/ESC、摄像头/图传三类。实际接入飞控后，该报告可由数传链路读取姿态角、驱动响应、转速、摄像头地址、帧率、链路延迟等数据生成。

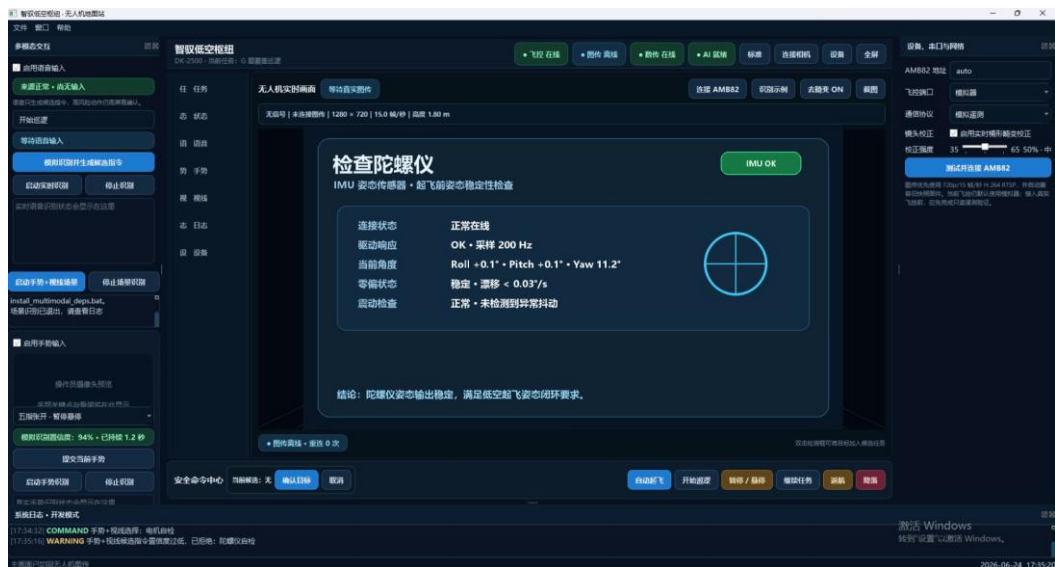


图 3-6 系统自检报告：陀螺仪、电机与摄像头状态

3.4.3 一键返航

返航流程用于在任务结束、低电量、链路异常或操作员主动召回时触发。返航同样属于高风险动作，需要候选指令与二次确认。确认后界面显示返航图传画面，并在右侧状态栏和底部日志中记录返航阶段。

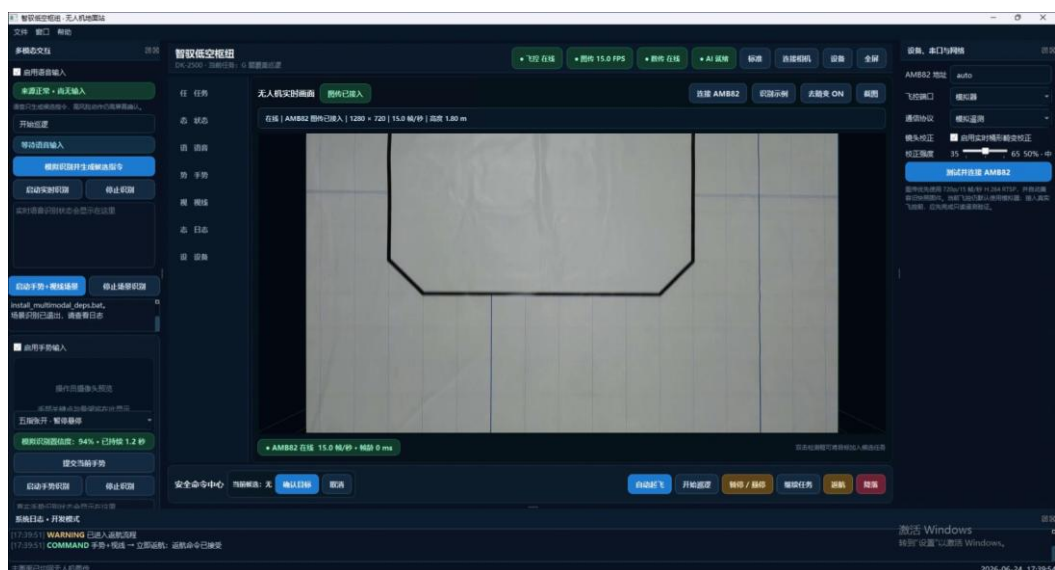


图 3-7 一键返航流程运行画面

3.5 功能五：巡线与火源识别

巡线功能围绕竞赛场地地图展开。任务地图将 48 dm × 40 dm 场地、障碍区、起飞区、返航区、航点与火源候选点统一显示。巡线过程中，地面站根据航线推进任务状态，并持续对图传画面执行火源检测。

火源识别算法以红/橙色高亮目标为主要检测对象，结合颜色阈值、亮度/饱和度、轮廓面积与连续帧确认机制过滤误检。当候选区域稳定出现后，主图传画面会显示红色检测框和置信度，底部事件中心高亮告警，任务地图中对应点位由蓝色变为红色并持续闪烁，同时右上角状态从“尚未发现火源”切换为“已发现火源”。

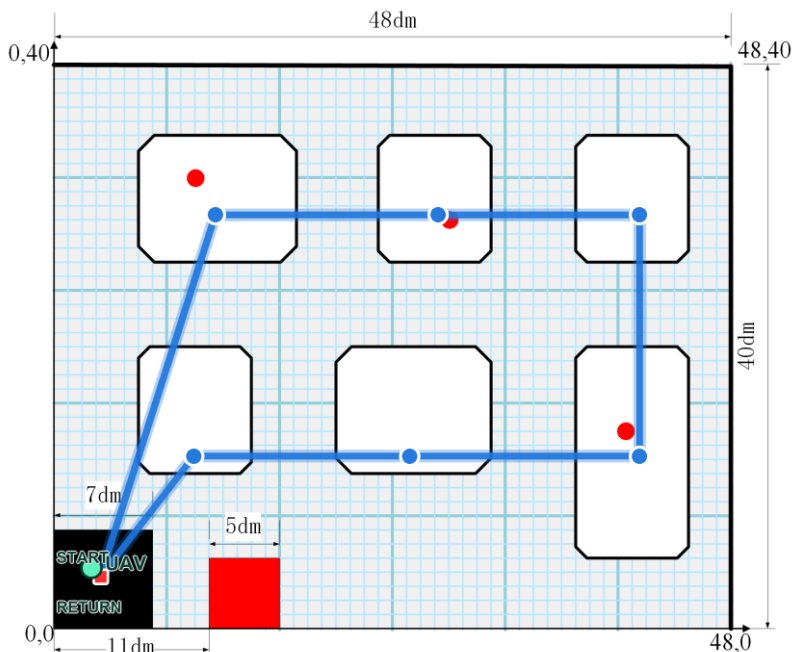


图 3-8 巡线路线与任务地图示意图

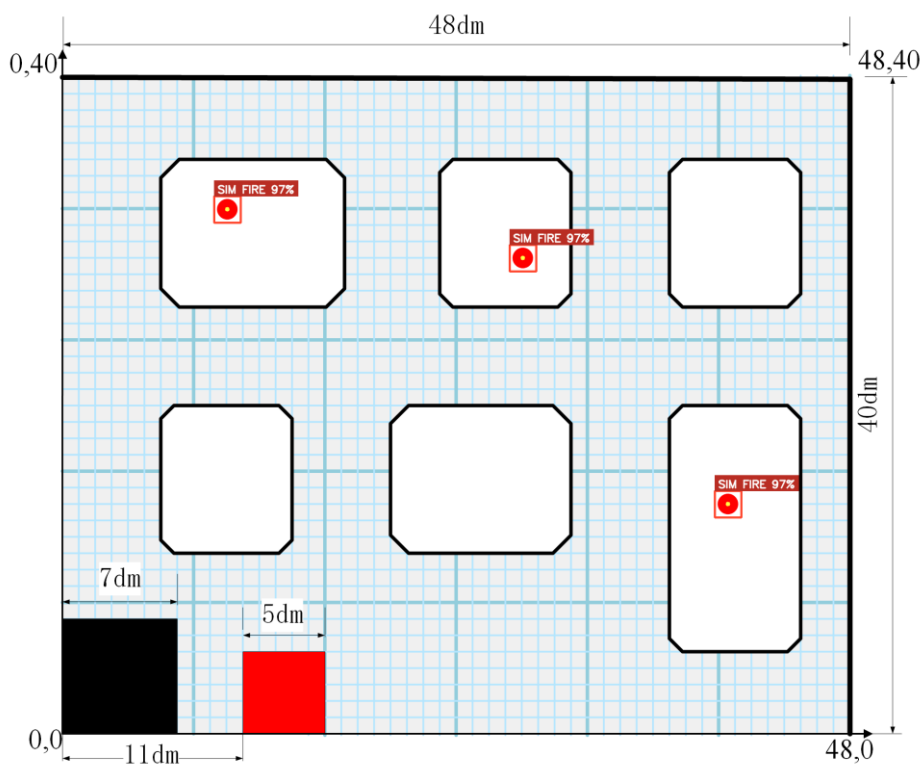


图 3-9 火源识别算法示例：红框圈出模拟火源

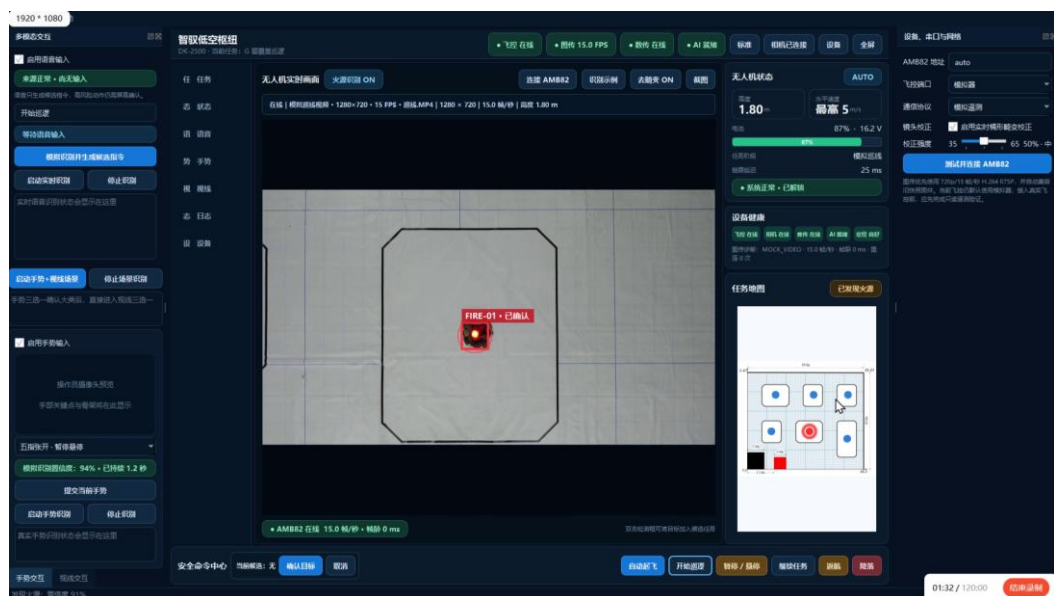


图 3-10 巡线过程中识别到火源

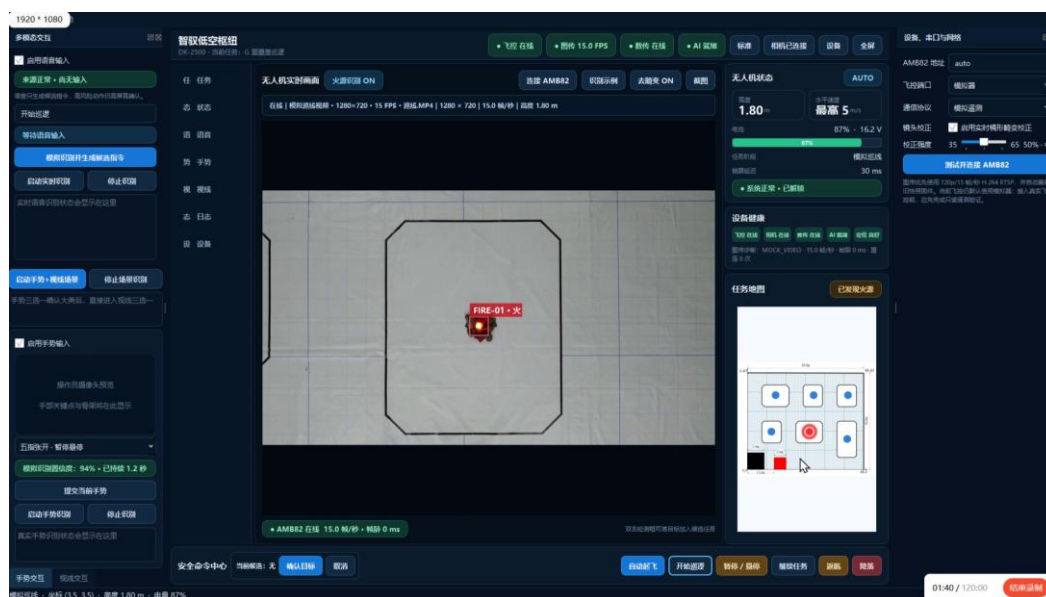


图 3-11 识别成功后任务地图标记火源位置

3.6 功能六：重要事项鼠标二次确认

为防止语音误识别、手势误触发或视线短时偏移造成危险动作，系统将高风险任务统一设计为“候选指令—人工确认—执行”的闭环。自动起飞、开始巡逻、返航、降落、确认火源等动作均不会在识别瞬间直接执行。

安全命令中心会显示当前候选动作、来源、置信度与风险等级。操作员需要通过鼠标点击“确认目标”或弹窗确认后，系统才会进入摄像头切换、任务执行和日志记录阶段；若选择取消，候选动作立即清空。

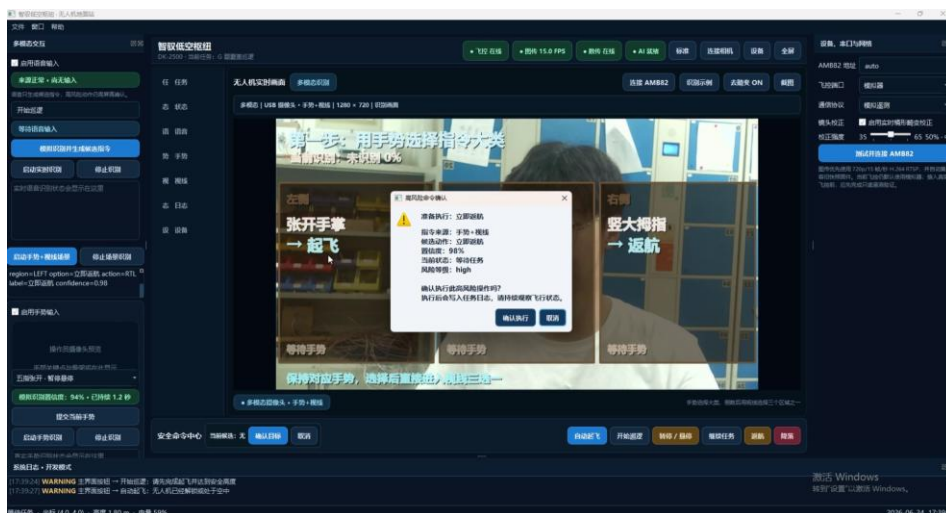


图 3-12 鼠标二次确认机制

3.7 功能七：地面站 UI 模块说明

地面站采用“一个主图传窗口 + 多个常驻状态/控制面板”的布局。摄像头画面保持主工作区地位，左侧负责多模态输入，右侧负责无人机状态、设备健康与任务地图，底部负责安全命令和系统日志。

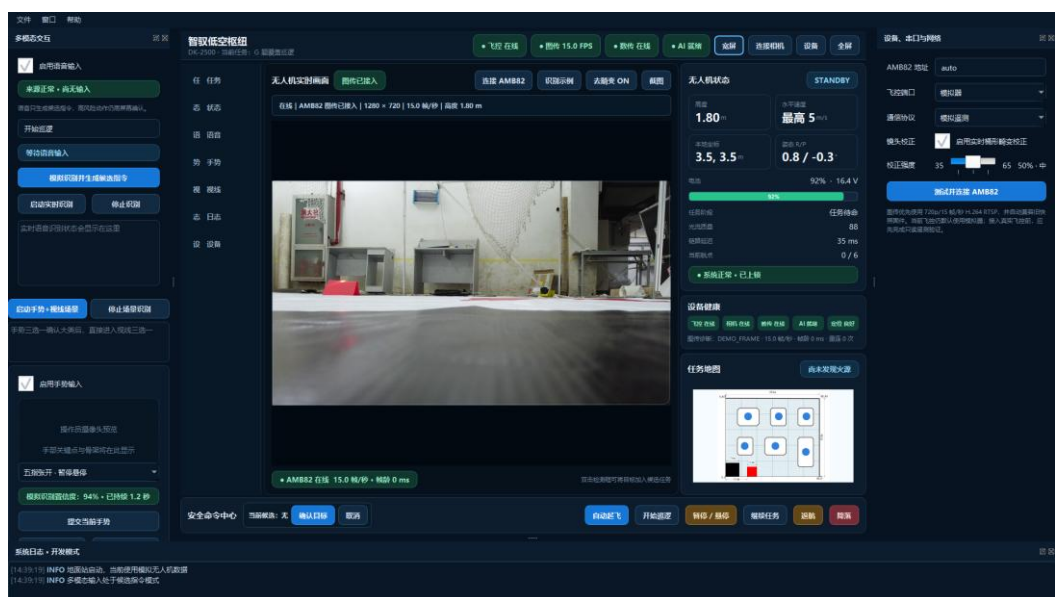


图 3-13 地面站整体运行界面

图 3-3 地面站各个模块及主要作用

模块	位置	主要作用
多模态交互区	左侧	语音、手势、视线输入；显示识别状态、候选动作和启动按钮
实时图传区	中央	显示 AMB82 图传、模拟视频、火源检测框、切换动画和自检报告
无人机状态区	右侧上方	显示高度、水平速度上限、电量、链路延迟、航点进度和任务阶段
设备健康区	右侧中部	显示飞控、相机、数传、AI、定位等模块在线状态
任务地图区	右侧下方	显示场地缩略图、航点、火源发现状态和巡线进度
设备与网络区	最右侧	配置 AMB82 地址、飞控端口、通信协议和镜头畸变校正强度
安全命令中心	底部	放置自动起飞、开始巡逻、暂停、继续、返航、降落等常用安全命令
系统日志区	底部展开区	记录启动、识别、告警、确认、任务阶段切换等事件

第四章 飞机平台、任务地图与巡逻路线设计

本章围绕无人机平台、任务地图、巡逻路线和数传链路展开说明。无人机端承担低空起降、航点巡线、图像采集和状态回传任务；地面站端承担多模态交互、图传显示、火源识别、任务地图和安全确认任务。二者通过图传与数传链路构成“地面站规划、飞控执行、状态回传、事件记录”的低空巡检闭环。

路线规划采用“地面站规划、飞控执行”的结构。地面站根据 $48\text{ dm} \times 40\text{ dm}$ 任务场地生成航点序列，并通过数传链路下发至飞控端；飞控端解析任务后，依次完成起飞、巡线、火源区域

覆盖、返航与降落流程。任务航点缓存、航点到达判定、速度/高度控制接口和数传确认机制共同支撑低空巡线任务执行。

4.1 无人机平台组成

虽然本项目以 Intel DK-2500 地面边缘枢纽为核心展示对象，但完整系统仍需要无人机作为低空任务执行节点。无人机部分主要承担飞行姿态控制、低空起降、巡线执行、图像采集和状态回传；地面站则承担多模态交互、图传显示、火源识别、任务地图和安全确认。二者结合后，系统才能形成“地面智能决策—空中执行反馈”的闭环。



图 4-1 F260-T432 四旋翼无人机平台整机示意图

图 4-1 展示了本作品使用的 F260-T432 四旋翼平台。该平台体量小、结构开放，适合承担低空巡线、图传回传和飞控任务执行。



图 4-2 F260-T432 无人机平台硬件组成框图

图 4-2 展示了飞控核心、惯性测量单元、电机与电调、AMB82 图传模块、数传链路和 Intel DK-2500 地面站之间的关系。无人机端并非孤立飞行器，而是地面站闭环中的任务执行端。

表 4-1 无人机平台硬件组成与作用说明

类别	硬件/模块	作用说明
飞控核心	F260-T432 飞控开发板	负责姿态解算、PID 控制、电机混控和起飞降落状态机。
惯性测量	陀螺仪	提供三轴角速度、加速度和姿态角，是自稳、定高和起飞保护的基础。
动力系统	四个无刷电机 + 电调 ESC	飞控输出 PWM 至电调，通过四电机差速实现升降、横滚、俯仰和偏航控制。
能源系统	锂电池与供电模块	为飞控、电调、电机和图传模块供电，并通过电压采集进行电量监测。
图传模块	AMB82-mini 摄像头模块	采集任务场地图像，回传至 DK-2500 地面站，用于实时显示与火源识别。
数传模块	nRF24L01 / UART 数传	在飞控、遥控器和地面端之间传输遥测数据与控制指令。

4.2 机载摄像模块与地面站图传关系

机载摄像模块与地面站之间通过无线局域网或局部网络完成图像回传。AMB82-mini 摄像头采集任务场地图像，地面站接收后在中央图传区域显示，并同步执行镜头去畸变、火源候选识别和事件记录。该链路保留了原报告中“图传接收—镜头校正—火源识别—地图回写”的处理逻辑，同时与无人机平台硬件组成对应起来。

图传结果不是单独用于观察画面，而是作为巡线任务中的感知输入：当无人机到达航点并悬停时，地面站读取当前图像帧，调用火源识别算法，并把识别结果写入任务地图和事件日志。这样，机载摄像头、地面站识别算法和飞控航点任务共同形成了“到点采集—视觉识别—状态回传—事件标记”的任务链。

4.3 起飞与降落的关键控制逻辑

飞控源码中的起飞与降落逻辑主要由三个层次组成：第一层是一键起飞/降落状态机，负责把操作指令转换为上升或下降速度；第二层是高度控制器，负责根据目标高度、当前高度、垂直速度和加速度计算油门输出；第三层是电机混控，把基础油门和姿态 PID 输出分配到四个电机。

在 `program\_ctrl.c` 中，`One\_Key\_Takeoff()` 先设置解锁标志，再置位一键起飞请求；`One\_Key\_Take\_off\_Land\_Ctrl\_Task()` 根据自动起降状态切换爬升、悬停和降落阶段。高度控制器位于 `height\_control.c`，其输出的基础油门随后进入 `MotorControl()`，与横滚、俯仰和偏航 PID 输出一起形成四旋翼 X 型混控。

代码 4-1 飞控起飞与高度控制逻辑节选

```
// 代码 4-1: 飞控起飞与高度控制逻辑节选
void One_Key_Takeoff(void)
{
    FC_CMD_UNLOCK;           // 设置解锁标志
    if (one_key_takeoff_f == 0)
        one_key_takeoff_f = 1; // 置位一键起飞请求
}

void Height_Control_Update(void)
{
    fb_vel = HeightInfo.Z_Speed;
    fb_hei = HeightInfo.Z_Postion;
}
```

// 代码 4-1：飞控起飞与高度控制逻辑节选

```
fb_acc = HeightInfo.Z_Acc;

exp_vel = ProgramCtrl.z_speed + auto_takeoff_speed;
HeightInfo.Thr = Height_PID_Calc(exp_vel, fb_vel, fb_hei, fb_acc);
}

void MotorControl(void)
{
    MOTOR1 = base_thr + RollPID - PitchPID + YawPID;
    MOTOR2 = base_thr + RollPID + PitchPID - YawPID;
    MOTOR3 = base_thr - RollPID + PitchPID + YawPID;
    MOTOR4 = base_thr - RollPID - PitchPID - YawPID;
}
```

代码 4-1 用于说明高层任务命令如何进入飞控控制链。地面站侧完成候选指令确认后，通过数传触发飞控端任务状态；飞控端再由高度控制和电机混控完成具体运动控制。

4.4 任务地图与路线规划实现

比赛场地可抽象为 $48\text{ dm} \times 40\text{ dm}$ 的二维平面坐标系，黑色区域为起飞/返航区域，红色区域为起飞区，白色多边形为障碍区域，红色圆点为模拟火源候选位置。地面站中的任务地图负责把场地坐标、航点、火源发现状态和巡线进度可视化。

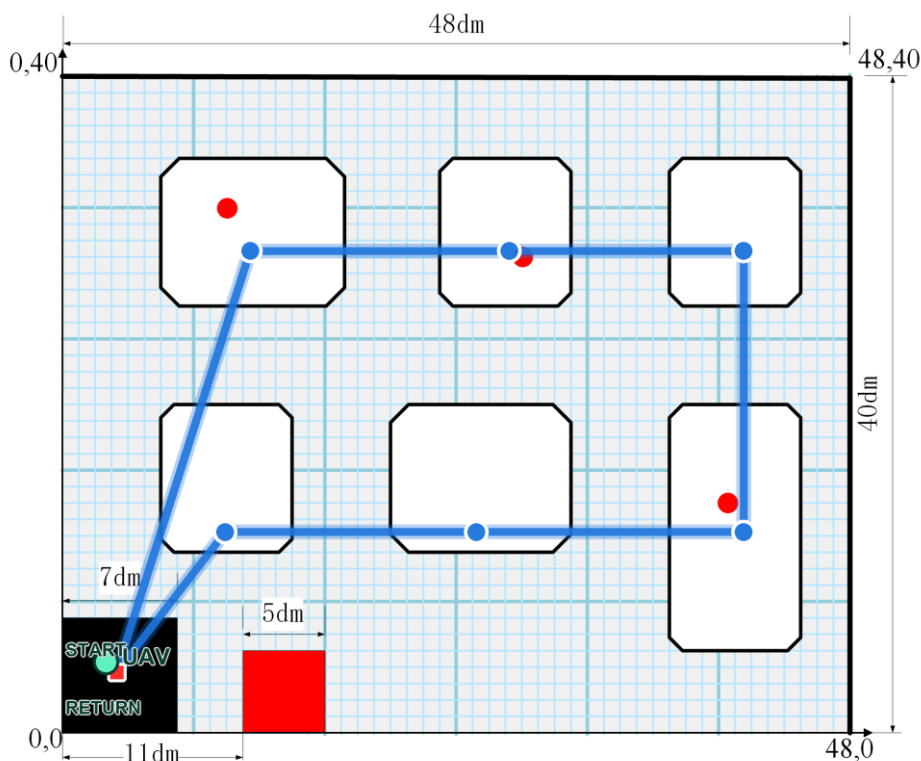


图 4-3 任务场地路径规划示意图

图 4-3 展示了任务场地路径规划方式。蓝色折线路径表示预设巡逻路线，白色多边形及其缓冲边界作为障碍禁区，红色圆点表示火源候选位置，用于说明航点序列如何覆盖场地以及火源事件如何在地图上定位。

路线规划采用“地面站规划、飞控执行”的结构。地面站根据比赛场地尺寸、障碍物位置和火源候选区域生成航点序列，并通过数传协议下发至飞控端。飞控端接收任务包后，将航点写入任务缓存，由任务状态机依次调用位置控制、高度控制和速度限制接口完成巡线飞行。每到

达一个航点，飞控会通过数传回传 ACK 与当前任务进度；当到达火源检测航点时，系统触发图传检测与地面站事件标记。该设计使路径规划、飞行控制、图传识别和地面站可视化形成完整闭环。

代码 4-2 默认火源巡检航点缓存定义

```
typedef struct {
    float x_dm;
    float y_dm;
    float target_alt_m;
    float speed_mps;
    uint8_t action;
} MissionWaypoint;

#define MAX_MISSION_WP 8

static MissionWaypoint mission_wp[MAX_MISSION_WP];
static uint8_t mission_wp_count = 0;
static uint8_t current_wp_index = 0;
static uint8_t mission_running = 0;

void Mission_LoadDefaultFirePatrol(void)
{
    mission_wp_count = 6;
    current_wp_index = 0;

    mission_wp[0] = (MissionWaypoint){11.5f, 29.0f, 1.8f, 1.2f, WP_PATROL};
    mission_wp[1] = (MissionWaypoint){27.0f, 29.0f, 1.8f, 1.2f, WP_PATROL};
    mission_wp[2] = (MissionWaypoint){41.0f, 29.0f, 1.8f, 1.2f, WP_PATROL};
    mission_wp[3] = (MissionWaypoint){41.0f, 12.0f, 1.8f, 1.2f, WP_PATROL};
    mission_wp[4] = (MissionWaypoint){25.0f, 12.0f, 1.8f, 1.0f, WP_FIRE_CHECK};
    mission_wp[5] = (MissionWaypoint){10.0f, 12.0f, 1.8f, 1.2f, WP_RETURN_PREP};
}
```

代码 4-2 展示了飞控侧任务航点缓存的定义方式。每个航点包含平面坐标、目标高度、飞行速度和动作类型，既能表达普通巡逻点，也能表达火源检测点和返航准备点。

4.5 航点执行状态机与火源识别联动

航点执行采用任务状态机实现。任务开始后，飞控端根据当前航点设置位置目标、高度目标和速度上限；当位置到达判定满足阈值时，系统进入下一航点。若当前航点属于火源检测区域，飞控通过数传向地面站发送到达事件，地面站同步调用图传识别模块，并将识别结果写入任务地图和日志。

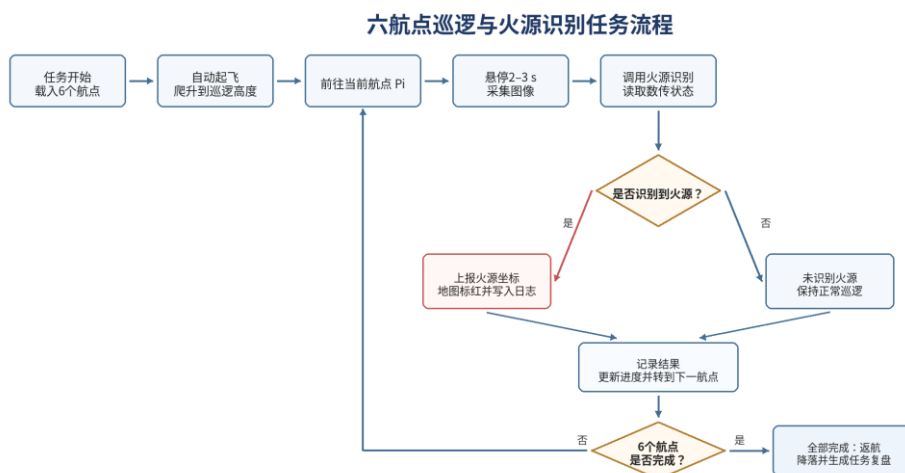


图 4-4 六航点巡逻与火源识别任务流程图

图 4-4 展示了六航点任务从开始、起飞、逐点巡逻、到点悬停、识别火源、记录结果到返航复盘的完整流程。该流程对应地面站任务地图、飞控航点状态机和事件日志三部分联动关系。

代码 4-3 航点执行状态机

```
void Mission_Task(float dt)
{
    if (!mission_running || current_wp_index >= mission_wp_count) {
        return;
    }

    MissionWaypoint *wp = &mission_wp[current_wp_index];

    Position_SetTarget(wp->x_dm, wp->y_dm);
    Height_SetTarget(wp->target_alt_m);
    Velocity_Limit(wp->speed_mps);

    if (Position_IsArrived(wp->x_dm, wp->y_dm, 0.5f)) {
        if (wp->action == WP_FIRE_CHECK) {
            Camera_SetDetectEnable(1);
            Telemetry_SendEvent(EVENT_REACH_FIRE_CHECK_AREA);
        }

        current_wp_index++;

        if (current_wp_index >= mission_wp_count) {
            Mission_StartReturnHome();
        }
    }
}
```

代码 4-3 展示了航点

执行状态机的核心过程：读取当前航点，调用位置、高度和速度控制接口，完成到达判定，并在火源检测航点触发图传识别事件。该逻辑与图 4-4 的流程相对应。

4.6 数传工作原理与飞控—地面站联动流程

数传链路的作用是连接地面站和飞控，使地面端能够获取无人机状态，也能够向无人机发送任务指令。结合现有代码和扩展设计，可以将数传分为物理链路、帧格式、消息内容和地面站解析四层。



图 4-5 MAVLink 数传协议栈分层示意图

在当前 F260-T432 源码中，`communication.c` 负责 nRF24L01 通信初始化、收发队列和端口选择，`ZKHD\_Link.c` 负责对上位机/遥控器/飞控之间的数据帧进行解析。数据到达后，`SwitchPort()` 判断来自 USB 还是 NRF 通道，并把有效负载分发到对应处理函数。

图 4-5 展示了数传协议栈分层思想：应用层负责航点、状态和命令语义，协议层负责帧格式与校验，传输层负责 UART / Wi-Fi / nRF24L01 等链路，硬件层负责 DK-2500、飞控和无线模块之间的数据通道。

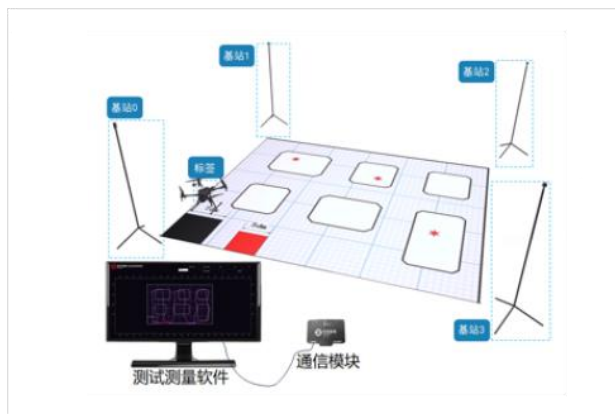


图 4-6 数传测试与上位机软件示意图

图 4-6 展示了数传测试与上位机软件界面，用于说明飞控端和地面端可通过上位机工具观察通信状态、任务进度和 ACK 回执。

表 4-2 数传链路分层与功能说明

层级	实现	主要功能
物理层	UART、USB、nRF24L01，无线图传独立通道	完成字节流收发、无线传输和端口选择。
协议层	ZKHD_Link、MAVLink	完成帧头、长度、消息 ID、校验和收发设备 ID 管理。
消息层	姿态、高度、电量、模式、起降、返航、航点	把二进制数据解释为飞控状态或控制指令。
应用层	地面站 UI、任务地图、事件日志、安全确认	显示无人机状态、生成候选命令并记录任务过程。

代码 4-4 数传任务包解析与 ACK 回传逻辑

```

void Telemetry_OnMissionPacket(uint8_t *payload, uint16_t len)
{
    MissionPacket packet;

    if (!DecodeMissionPacket(payload, len, &packet)) {
        Telemetry_SendAck(ACK_MISSION_ERROR);
        return;
    }

    for (uint8_t i = 0; i < packet.count && i < MAX_MISSION_WP; i++) {
        mission_wp[i].x_dm = packet.wp[i].x_dm;
        mission_wp[i].y_dm = packet.wp[i].y_dm;
        mission_wp[i].target_alt_m = packet.wp[i].alt_m;
        mission_wp[i].speed_mps = packet.wp[i].speed_mps;
        mission_wp[i].action = packet.wp[i].action;
    }

    mission_wp_count = packet.count;
    current_wp_index = 0;
    Telemetry_SendAck(ACK_MISSION_READY);
}

```

代码 4-4 展示了地面站下发航点任务包后的飞控端解析逻辑。飞控端首先校验任务包，随后把航点坐标、高度、速度和动作类型写入任务缓存，并通过 ACK 告知地面站任务已准备完成。该过程使航线规划、飞控执行和地面站状态显示能够同步。

综上，第四章通过无人机平台、硬件组成、任务路径和数传链路说明飞行端在系统中的作用，并通过航点缓存、任务状态机和数传任务包解析代码补足实现细节。无人机不是地面站的外部展示对象，而是低空巡检任务闭环中的执行主体。

第五章 地面站与飞控闭环关键技术实现

5.1 地面站主窗口与任务总线

地面站主窗口基于 PyQt5 组件化实现，采用顶部状态栏、中央图传区、侧边状态区、任务地图、安全命令中心和底部日志区组合布局。主窗口不只承担界面显示功能，还通过任务总线把图传线程、火源识别、多模态输入、数传接口和事件日志连接起来，使不同输入来源生成的候选命令进入统一处理流程。

任务总线采用“输入来源—候选意图—风险分级—安全确认—飞控执行—状态回传—事件记录”的顺序组织系统逻辑。语音、手势、视线和面板按钮只负责生成候选意图，真正的执行入口集中在安全命令中心，从结构上避免多入口直接控制无人机造成的误触发风险。

表 5-1 关键软件模块与闭环作用

模块	主要输入	主要输出	闭环作用
图传线程	AMB82 RTSP 视频流	实时图像帧、帧率、连接状态	为火源识别和人工观察提供感知输入
火源识别模块	校正后图像帧	候选目标、置信度、检测框	把视觉目标转化为可确认事件
多模态交互模块	语音、手势、视线、按钮	候选命令、来源、风险等级	降低操作门槛并统一进入安全流程
安全命令中心	候选目标与候选命令	确认结果、执行指令、取消记录	防止高风险命令被直接接触发
数传任务接口	确认后的任务命令	任务包、ACK、状态回传	连接地面站决策与飞控执行
事件日志模块	识别、确认、执行、回执事件	时间戳、事件类型、结果	支撑全过程复盘与评审展示

5.2 AMB82 图传接入、镜头校正与火源检测

AMB82 图传以 RTSP 视频流方式接入地面站，视频读取在独立线程中完成。线程读取图像后先进行镜头校正，再调用火源检测模块，并把原始帧、检测框、帧率和连接状态同步发送到主界面。该设计把图像采集和界面响应分离，避免视频读取阻塞安全按钮、状态刷新和日志写入。

火源识别采用颜色、亮度、饱和度和轮廓面积联合判断，面向竞赛场景中的可见火源或红橙色模拟火源标志。算法输出候选区域后，系统不会立即写入正式事件，而是进入连续帧确认与人工确认流程，从而降低反光、红色背景和瞬时噪声造成的误报。

表 5-2 图传与识别处理链路

处理阶段	实现方式	输出结果	工程作用
视频接入	AMB82 RTSP	实时画面、连接状态	保证主工作区持续可见
镜头校正	畸变参数与画面裁切	比例稳定的图像帧	降低边缘形变对识别和观察的影响
目标检测	颜色阈值、亮度/饱和度、轮廓面积	候选火源框与置信度	生成可解释的视觉候选目标
连续确认	连续 3 帧超过阈值才触发	稳定候选事件	过滤瞬时噪声和单帧误报
界面叠加	检测框、置信度、事件提示	视频区与事件区同步更新	让操作者快速定位目标

5.3 连续帧确认与候选目标生成

为避免单帧误检直接影响任务判断，系统设置连续帧确认机制。检测模块每帧输出候选框和置信度，确认器只在目标连续 3 帧达到阈值时生成候选事件；若目标消失或置信度低于阈值，连续计数自动清零。该机制使火源告警从“单帧检测结果”升级为“稳定候选目标”。

候选目标生成后，主画面显示检测框和置信度，任务地图对应点位切换为告警状态，最近事件区记录候选来源。操作者点击确认后，候选目标才写入正式事件日志，并作为后续返航、悬停或复核流程的依据。

代码 5-1 连续帧确认逻辑

```
class FireConfirmationTracker:
    def __init__(self, required_frames=3, confidence_threshold=0.68):
        self.required_frames = required_frames
        self.confidence_threshold = confidence_threshold
        self.streak = 0

    def update(self, detections):
        top = detections[0] if detections else None
        if top and top.confidence >= self.confidence_threshold:
            self.streak += 1
        else:
            self.streak = 0
        return top if self.streak >= self.required_frames else None
```

5.4 多模态候选指令生成机制

表 5-3 多模态输入到候选指令的统一映射

输入方式	识别结果	进入系统后的表示	是否直接执行
语音输入	开始巡逻、起飞、返航等文本意图	CommandIntent	否，进入候选命令
手势输入	张开手掌、握拳、竖大拇指	任务大类选择	否，等待子选项选择
视线/头部方向	左、中、右卡片选择	具体子任务选择	否，高风险动作仍需确认
面板按钮	明确按钮动作	候选命令或低风险操作	高风险动作需确认

多模态输入采用统一候选指令模型。语音输入负责把自然语言命令转换为任务意图；手势输入负责选择任务大类；视线或头部方向负责在左、中、右三张候选卡片中选择子任务；面板按钮作为调试和应急兜底入口。四类输入均不直接绕过安全命令中心，而是写入同一套候选命令结构。

手势/视线交互采用“三阶段”机制：第一阶段识别手势并确定任务大类，第二阶段由视线或头部方向选择左、中、右子选项，第三阶段由安全命令中心完成高风险动作二次确认。该逻辑使多模态交互具有展示性，同时保留明确的人工执行权。

5.5 安全命令中心与二次确认状态机

安全命令中心是系统执行权的唯一入口。候选目标或候选命令进入命令中心后，系统根据动作类型标记风险等级。截图、提示、普通日志属于低风险事件，可直接记录；起飞、开始巡线、返航、降落、确认火源等动作属于高风险或任务关键动作，必须经过鼠标或按钮二次确认后才向飞控任务接口发送执行指令。

状态机由 IDLE、PENDING、CONFIRMED、EXECUTING、ACKED 和 CANCELED 六个状态组成。候选命令生成后进入 PENDING；操作者确认后进入 CONFIRMED；地面站发送任务包后进入 EXECUTING；飞控回传 ACK 后进入 ACKED；若操作者取消或超时，命令进入 CANCELED 并写入日志。

代码 5-2 安全命令状态机核心逻辑

```
def handle_command(intent):
    show_pending_command(intent)
    if intent.risk_level in ["high", "mission"]:
        if not user_confirmed(intent):
            log_event("CANCELED", intent.action)
            return
    packet = build_flight_task_packet(intent)
    telemetry.send(packet)
    ack = telemetry.wait_ack(timeout=1.0)
    update_task_state(intent.action, ack)
    log_event("ACKED", intent.action)
```

5.6 数传任务接口与飞控状态回传

数传任务接口连接地面站和飞控任务状态机。地面站在确认高风险命令后，将起飞、巡线、悬停、返航、降落和火源复核等动作封装为任务包；飞控端解析任务包后更新任务状态，并通过 ACK、航点进度、电量、链路状态和任务阶段等字段回传执行结果。

该接口使地面站不再只是显示界面，而是能够完成“任务生成—指令下发—状态回传—日志记录”的完整闭环。任务地图根据飞控回传的当前航点和任务阶段刷新，事件日志记录每次命令发送、确认、ACK 和异常提示。

表 5-4 地面站—数传—飞控任务接口

任务命令	地面站动作	飞控侧响应	回传信息
TAKEOFF	生成起飞任务包并等待确认	进入起飞状态机	起飞 ACK、高度状态
START_MISSION	下发巡线航点序列	写入航点缓存并执行巡线	当前航点、任务进度
HOLD	下发悬停命令	保持当前位置或任务点	悬停状态、姿态信息
RTL	下发返航命令	进入返航航点或 HOME 流程	返航 ACK、距离/阶段
LAND	下发降落命令	进入降落状态机	降落 ACK、电机状态

5.7 任务地图、事件日志与可追溯记录

任务地图以 48 dm × 40 dm 场地为基础，显示起飞区、返航区、障碍区、预设航点、火源候选点和当前任务进度。每当飞控回传航点到达或火源检测模块生成候选事件时，地图同步刷新点位状态，使操作者能够同时看到“无人机在哪里、任务进行到哪一步、是否发现火源”。

事件日志记录系统启动、图传连接、候选命令、人工确认、任务包发送、飞控 ACK、火源告警和返航/降落等关键节点。每条日志包含时间、事件来源、动作类型和结果，支撑评审现场复盘，也便于赛后分析误报、延迟和操作链路。

5.8 完整闭环 workflow 实现

综合上述模块，本系统形成“图传感知—火源候选—多模态命令—安全确认—飞控执行—状态回传—事件记录”的工程闭环。图传和识别模块负责发现目标，多模态输入负责生成任务意图，安全命令中心负责人工确认，飞控任务接口负责执行动作，任务地图和日志负责反馈与留痕。

该闭环使系统不再停留在单独算法或单独界面展示层面，而是把目标发现、任务决策、飞行执行和结果记录组织为一套可解释、可演示、可复盘的人机协同控制流程。

第六章 系统闭环测试与结果分析

6.1 测试环境与评价指标

系统测试围绕图传链路、火源识别、多模态候选命令、安全命令中心、数传任务接口、地图巡线和事件日志展开。测试目标不是单独验证某个界面控件，而是验证系统能否稳定完成从目标发现到飞控执行再到状态回传的完整闭环。

表 6-1 闭环测试环境

项目	配置或状态	评价指标
地面站主机	Windows + Python + PyQt5 + OpenCV	界面响应、日志记录、任务状态刷新
图传模块	AMB82 RTSP/快照回退	帧率、延迟、连接恢复
识别算法	颜色/亮度/轮廓特征 + 连续帧确认	候选框、置信度、误触发抑制
多模态输入	OpenVINO Whisper、手势、视线/头部方向、按钮	候选命令生成时间、确认流程
飞控任务接口	任务包、ACK、航点状态、电量/链路状态	命令回执、任务进度、异常记录

6.2 图传链路 & 界面响应测试

图传测试通过连接 AMB82 视频流，观察实时画面、帧率、帧龄、画面比例和连接异常处理。测试结果表明，视频区能够保持稳定刷新，图传线程不会阻塞安全按钮、地图刷新和系统日志输出。

表 6-2 图传链路与界面响应测试结果

测试项	测试方法	实测结果	结论
相机连接	启动地面站并连接 AMB82 RTSP	显示 ONLINE，视频稳定进入主画面	图传接入稳定
帧率	连续观察视频状态栏	约 15 FPS	满足巡检演示需求
链路延迟	观察帧龄与状态刷新	约 31 ms	可支撑实时观察和识别叠加
画面比例	切换不同窗口尺寸	主画面保持 16:9，不明显拉伸	显示逻辑正确
异常处理	断开或输入错误地址	界面保持运行并输出错误日志	具备容错能力

6.3 火源识别与连续确认测试

火源识别测试使用示例图、红橙色模拟火源目标和实时视频画面进行验证。系统在检测到候选区域后显示检测框和置信度，并通过连续 3 帧确认机制过滤瞬时噪声。确认后的候选目标同步写入最近事件区和任务地图。

表 6-3 火源识别与连续确认测试结果

测试项	测试方法	实测结果	结论
候选框输出	输入含火源标志的示例图	可输出红色检测框与置信度	检测链路有效
实时画面识别	在视频画面中放置红橙色目标	置信度约 92%~94%	受控场景下识别稳定
连续帧确认	短时遮挡或快速移走目标	连续 3 帧后才生成候选事件	降低单帧误报
地图联动	确认候选目标后观察任务地图	对应点位变为告警状态	识别结果可定位
事件记录	触发火源候选与确认	日志记录时间、来源、置信度和结果	具备可追溯性

6.4 多模态候选命令测试

多模态交互测试重点验证输入是否能统一转化为候选命令，而不是绕过安全流程直接控制飞行。语音、手势、视线和面板按钮均进入 CommandIntent 结构，并由安全命令中心显示来源、动作、置信度和风险等级。

表 6-4 多模态候选命令测试结果

输入方式	测试内容	实测结果	闭环表现
语音输入	说出“开始巡逻”“返航”等命令	约 1.0~1.8 s 生成候选命令	文本意图进入安全命令中心
手势输入	张开手掌、握拳、竖大拇指	稳定保持约 1.2 s 后确定任务大类	手势只选择大类，不直接执行
视线/头部方向	在三张卡片中选择左/中/右选项	约 1 s 内完成子选项定位	子任务进入候选命令
面板按钮	点击起飞、巡线、返航、降落	候选命令即时生成	高风险动作仍需确认

6.5 安全命令与飞控任务闭环测试

安全命令测试覆盖自动起飞、开始巡线、暂停/悬停、继续任务、返航和降落等动作。测试时先由多模态输入或面板按钮生成候选命令，再由安全命令中心确认，随后通过数传任务接口发送至飞控任务状态机，并等待 ACK 或状态回传。

表 6-5 安全命令与飞控任务闭环测试结果

测试项	测试方法	实测结果	结论
起飞命令	生成 TAKEOFF 候选并确认	任务包发送，收到起飞 ACK	起飞闭环成立
开始巡线	START_MISSION 并下发航点	航点状态进入执行流程	巡线任务可触发
暂停/继续	巡线过程中触发 HOLD / CONTINUE	状态栏与日志同步变化	任务状态切换清晰
返航命令	确认 RTL 高风险命令	返航事件写入日志并回传状态	安全策略有效
降落命令	确认 LAND 高风险命令	降落 ACK 与日志记录同步生成	闭环记录完整
取消命令	候选阶段点击取消	命令不下发，日志记录 CANCELED	误触发可被拦截

6.6 地图巡线与状态回传测试

地图巡线测试用于验证 48 dm × 40 dm 任务场地、航点推进、火源候选点和飞控回传状态之间的对应关系。地面站根据预设航点序列显示巡线路径，飞控任务状态机回传当前航点和任务阶段后，地图、状态栏和事件日志同步刷新。

表 6-6 地图巡线与状态回传测试结果

测试项	测试方法	实测结果	结论
任务地图	加载 48 dm × 40 dm 场地	场地边界、障碍区、航点和 HOME 点显示正确	地图表达准确
航点推进	执行六航点巡线任务	当前航点随任务阶段更新	路线逻辑清晰
状态回传	观察高度、电量、链路和任务阶段	状态区与日志持续刷新	飞控反馈可见
火源联动	到达火源检测航点并触发识别	地图点位与事件中心同步告警	识别与地图联动成立
刷新时延	观察地图和事件区更新	状态更新时间小于 1 s	满足任务展示需求

6.7 测试结果汇总与系统应用价值

综合测试结果表明，系统已经完成图传接入、目标识别、多模态候选命令、安全确认、飞控任务接口、状态回传、任务地图和事件日志的闭环联动。各模块不再是分散功能点，而是围绕低空巡检任务形成连续流程。

表 6-7 系统闭环测试结果汇总

模块	关键指标	测试结果	完成状态
图传链路	帧率、延迟、异常处理	约 15 FPS，延迟约 31 ms，断连可记录	完成
火源识别	置信度、连续确认、日志记录	置信度约 92%~94%，连续 3 帧确认	完成
多模态输入	候选命令生成与风险分级	语音、手势、视线、按钮均进入候选流程	完成
安全确认	高风险动作二次确认	起飞、巡线、返航、降落均需确认	完成
飞控任务接口	任务包、ACK、状态回传	任务命令可下发并回传执行状态	完成
任务地图	航点、火源、任务阶段刷新	状态更新时间小于 1 s	完成
事件日志	目标、命令、回执、异常记录	关键节点均有时间戳和结果	完成

表 6-8 系统应用价值对比表

维度	传统专业飞手/商业一体化方案	本作品方案	应用价值
成本	依赖高价行业无人机和专业配套设备	低成本模块化飞行端 + DK-2500 地面智能枢纽	适合教学、原型验证和轻量巡检
培训门槛	需要熟悉遥控器杆量、飞控模式和航线设置	通过语音、手势、视线三卡片和面板按钮进行任务级选择	降低非专业人员入门门槛
操作步骤	起飞、自检、巡线、火源记录等环节较分散	预封装预热、自检、巡线、识别、返航和降落流程	降低危险场景下的认知负担
安全确认	高度依赖飞手经验和现场判断	高风险动作进入候选展示、风险分级和二次确认	降低误触发与误执行风险
可维护性	一体机维修成本和周期较高	电机、电调、机架、图传、数传等可单独替换	适合反复调试与教学演示
可追溯性	过程记录依赖人工截图或口头汇报	任务地图、设备状态、事件日志和系统日志同步记录	便于复盘、评审展示和优化
扩展性	功能与厂商生态深度绑定	地面站可扩展模型、传感器、飞控协议和任务地图	利于持续迭代和二次开发

表 6-8 表明，本系统的价值不在于与工业级救援无人机进行性能替代，而在于通过低成本模块化飞行端、任务级交互和安全可追溯闭环，为教学竞赛、原型验证和轻量化低空巡检提供可落地的系统方案。

6.8 稳定性分析与扩展方向

从闭环测试结果看，系统的稳定性主要来自三个设计：第一，图传、识别和语音识别均采用独立线程或独立进程，避免计算任务阻塞主界面；第二，所有高风险动作集中进入安全命令中心，保证执行前有明确人工确认；第三，数传任务接口和事件日志同步记录任务下发、ACK回传和状态变化，使任务过程可复盘。

在扩展方向上，系统可进一步增加更多火源样本、复杂光照场景和多路线任务，以提升算法泛化能力和任务覆盖范围；也可继续扩展热成像、定位模块和更多飞控协议。上述扩展属于应用场景深化，不影响当前系统已经形成的地面站—数传—飞控闭环。

第七章 总结与展望

本文面向低空巡检与火源识别任务，完成了一套多模态无人机地面站系统的设计与实现。系统以地面站为核心，将 AMB82 图传、火源识别、任务地图、多模态交互、安全命令、数传任务接口和事件日志组织为完整任务链，形成“目标发现—人工确认—飞控执行—状态回传—事件记录”的人机协同控制闭环。

本作品的系统亮点可概括为四个方面：一是低成本模块化飞行端，将昂贵的一体化能力拆分为“低成本飞行端 + 地面智能枢纽”；二是低培训门槛的人机交互，通过语音、手势、视线三卡片和面板按钮把复杂飞控操作转化为任务级选择；三是预封装任务命令与快速响应，将预热、自检、低空起飞、巡线、火源识别、返航和降落封装为标准流程；四是安全可追溯闭环，所有高风险命令均经过候选生成、人工确认、执行反馈和日志记录，兼顾智能交互效率与无人机操作安全。

面向更复杂的应用场景，系统可继续扩展更多火源样本、复杂光照测试、热成像传感器、多机协同与更丰富的飞控协议。通过增加实测场景和任务路线，系统能够进一步提升工程鲁棒性和竞赛展示说服力。

参考文献

- [1] MAVLink Project. MAVLink Developer Guide[EB/OL]. [2026-06-30]. <https://mavlink.io/en/>.
- [2] PX4 Development Team. PX4 User and Developer Guide: MAVLink Messaging[EB/OL]. [2026-06-30]. <https://docs.px4.io/main/en/mavlink/>.
- [3] ArduPilot Development Team. MAVLink Basics[EB/OL]. [2026-06-30]. <https://ardupilot.org/dev/docs/mavlink-basics.html>.
- [4] QGroundControl Project. QGroundControl User Guide[EB/OL]. [2026-06-30]. <https://docs.qgroundcontrol.com/>.
- [5] Intel Corporation. OpenVINO Toolkit Documentation[EB/OL]. [2026-06-30]. <https://docs.openvino.ai/>.
- [6] Intel Corporation. Intel Distribution of OpenVINO Toolkit[EB/OL]. [2026-06-30]. <https://www.intel.com/content/www/us/en/developer/tools/openvino-toolkit/overview.html>.
- [7] OpenCV Team. OpenCV Documentation[EB/OL]. [2026-06-30]. <https://docs.opencv.org/>.
- [8] The Qt Company. Qt for Python Documentation[EB/OL]. [2026-06-30]. <https://doc.qt.io/qtforpython/>.
- [9] Riverbank Computing. PyQt5 Reference Guide[EB/OL]. [2026-06-30]. <https://www.riverbankcomputing.com/static/Docs/PyQt5/>.
- [10] Radford A, Kim J W, Xu T, et al. Robust Speech Recognition via Large-Scale Weak Supervision[C]//Proceedings of ICML. 2023.
- [11] Redmon J, Divvala S, Girshick R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]//CVPR. 2016: 779-788.
- [12] Bochkovskiy A, Wang C Y, Liao H Y M. YOLOv4: Optimal Speed and Accuracy of Object Detection[J/OL]. arXiv:2004.10934, 2020.
- [13] Reis D, Kupec J, Hong J, et al. Real-Time Flying Object Detection with YOLOv8[J/OL]. arXiv:2305.09972, 2023.
- [14] Shamsoshoara A, Afghah F, Razi A, et al. Aerial Imagery Pile Burn Detection Using Deep Learning: The FLAME Dataset[J]. Computer Networks, 2021, 193: 108001.
- [15] Mukhiddinov M, Abdusalomov A B, Cho J. A Wildfire Smoke Detection System Using UAV Images Based on the Optimized YOLOv5[J]. Sensors, 2022, 22(23): 9384.
- [16] Saydirasulovich S N, Mukhiddinov M, Djuraev O, et al. An Improved Wildfire Smoke Detection Based on YOLOv8 and UAV Images[J]. Sensors, 2023, 23(20): 8374.
- [17] Hopkins B H, O'Neill L, Marinaccio M, et al. FLAME 3 Dataset: Radiometric Thermal UAV Imagery for Wildfire Management[J/OL]. arXiv:2412.02831, 2024.
- [18] Meleti U, Razi A. Obscured Wildfire Flame Detection by Temporal Analysis of Smoke Patterns Captured by UAS[J/OL]. arXiv:2307.00104, 2023.
- [19] Haque S R, Mohammad N, Chowdhury M, et al. Drone Ground Control Station with Enhanced Safety Features[C]//IEEE Region 10 Humanitarian Technology Conference. 2017.
- [20] Kilic F, Janson N, Fabian B, et al. Prototype for Multi-UAV Monitoring-Control System Using WebRTC-Based GCS[J]. Drones, 2024, 8(10): 551.

附录 A 系统软件总体结构说明

本作品的软件系统围绕“多模态输入—地面站决策—无人机任务执行—状态回传—安全确认”这一闭环展开。整体软件由地面站主程序、图传接入模块、火源识别模块、多模态识别模块、任务状态机、事件日志模块、飞控通信接口和地图可视化模块组成。

系统软件采用模块化设计，各模块之间通过统一的数据模型和事件回调机制进行交互。地面站负责接收语音、手势、视线、鼠标按钮等多种输入方式，将原始输入转换为候选任务指令；候选任务经过安全确认后，由任务状态机生成飞控指令，并通过数传链路发送至无人机端。无人机端执行起飞、巡线、自检、返航、降落等动作后，将高度、电量、姿态、任务阶段、链路状态等信息回传至地面站，形成完整的人机协同控制流程。

软件总体结构如下：

层级	模块	主要功能
交互层	语音识别、手势识别、视线识别、鼠标按钮	接收操作者意图，生成候选任务
决策层	候选指令管理、安全二次确认、任务状态机	判断任务合法性，防止误触发
感知层	AMB82 图传、火源识别、镜头畸变校正	获取无人机视角画面并识别目标
执行层	飞控任务接口、MAVLink/串口通信、任务封装	将任务转换为飞控可执行命令
可视化层	主画面、任务地图、状态面板、事件中心	显示图传、地图、告警、任务状态
记录层	事件日志、任务记录、识别结果缓存	保存关键过程，便于复盘和调试

附录 B 地面站程序文件结构

地面站程序按照“主界面、功能模块、算法模块、资源文件、运行脚本”进行组织，便于后续维护和扩展。

```
ground_station/
├─ main.py           # 地面站主程序入口，负责窗口初始化与全局调度
├─ widgets.py        # UI 组件、状态面板、任务地图、事件中心
├─ camera_service.py # 图传接入、视频帧读取、RTSP/快照流管理
├─ fire_detector.py   # 火源识别算法，包括颜色阈值、连通域分析和连续帧确认
├─ lens_correction.py # 镜头畸变校正模块，支持多档校正强度
├─ models.py          # 无人机状态、任务状态、事件记录等数据模型
├─ simulator.py       # 模拟无人机状态、模拟任务流程与演示数据生成
├─ styles.py          # UI 样式表、字体、颜色、按钮状态
├─ requirements.txt   # Python 依赖列表
├─ start_ground_station.bat # 地面站启动脚本
├─ examples/          # 示例图片、地图、演示素材
└─ README.md          # 使用说明与运行方法
```

多模态识别模块独立放置，方便在不同电脑上单独调试：

```
multimodal_recognition/
├─ gesture_recognition.py # 三类手势识别：张开手掌、握拳、竖大拇指
├─ gaze_recognition.py    # 头部姿态 + 视线方向识别
├─ voice_asr.py           # 语音识别与候选指令生成
├─ run_gesture_recognition.bat
├─ run_gaze_recognition.bat
└─ install_multimodal_deps.bat
```

附录 C 地面站主程序运行流程

地面站启动后首先初始化 UI、加载地图资源、建立无人机状态模型，并进入事件循环。系统运行过程中，所有输入均不会直接控制无人机，而是先转化为“候选任务”，再交由安全确认机制处理。

地面站主流程如下：

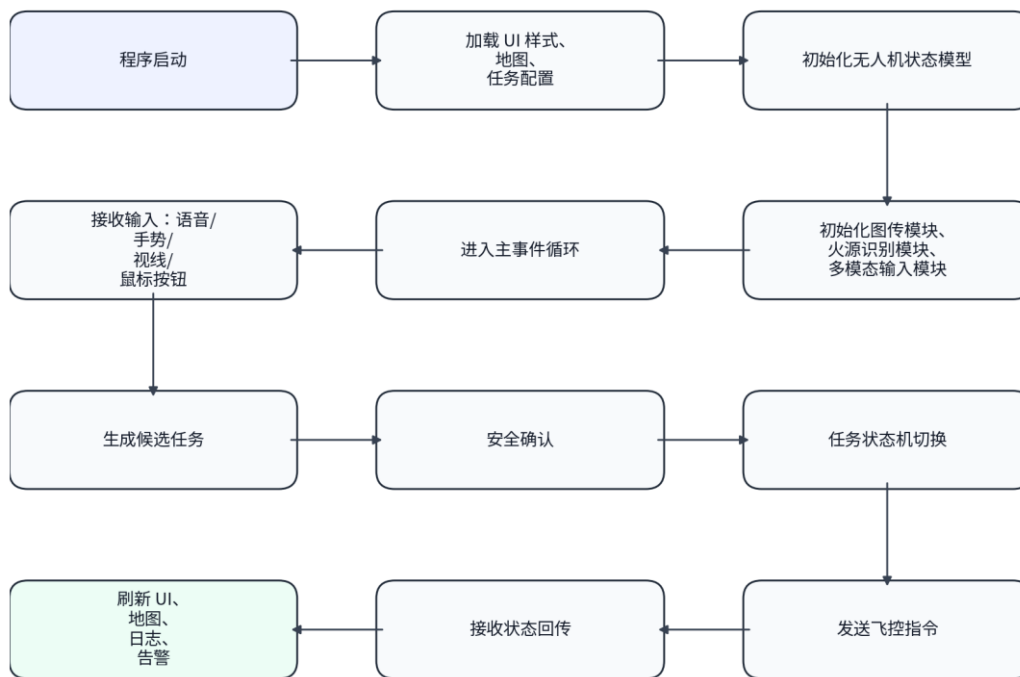


图 C 地面站主程序运行流程图

核心思想是将“识别”和“执行”解耦。识别模块只负责判断操作者意图，执行模块必须经过候选指令、安全确认和任务状态机三层过滤，从而降低误识别带来的风险。

附录 D 无人机状态数据模型

地面站内部使用统一的数据模型保存无人机状态。该模型既可以接收真实飞控回传，也可以在演示环境下接收模拟数据，保证 UI 显示和任务逻辑的一致性。

主要字段如下：

字段	含义
armed	无人机是否解锁
mode	当前飞行模式，例如 STANDBY、AUTO、RETURN
altitude	当前高度
velocity	当前水平速度
battery_percent	电池百分比
battery_voltage	电池电压
roll / pitch / yaw	姿态角
link_delay	通信链路延迟
camera_status	图传状态
ai_status	AI 识别模块状态
mission_stage	当前任务阶段
current_waypoint	当前航点编号
fire_detected	是否发现火源
fire_position	火源在地图中的位置

数据结构如下：

```
@dataclass
class DroneState:
    armed: bool = False
    mode: str = "STANDBY"
    altitude: float = 0.0
    velocity: float = 0.0
    battery_percent: int = 92
    battery_voltage: float = 16.4
    roll: float = 0.0
    pitch: float = 0.0
    yaw: float = 0.0
    link_delay_ms: int = 32
    camera_online: bool = False
    ai_ready: bool = True
    mission_stage: str = "任务待命"
    current_waypoint: int = 0
    total_waypoints: int = 6
    fire_detected: bool = False
    fire_position: tuple | None = None
```


附录 E 图传接入与画面处理流程

图传模块负责接入 AMB82 摄像头画面。系统支持 RTSP 视频流和 snapshot 快照流两种形式，并优先使用 720p / 15 FPS 的 H.264 RTSP 图传，以兼顾清晰度、延迟和稳定性。图传处理流程如下：



图 E 图传接入与画面处理流程图

为适应 AMB82 广角镜头带来的球面畸变，系统加入了实时去畸变模块。地面站提供 35、50、65 三档校正强度，操作者可以根据摄像头安装角度和现场画面选择合适档位。校正后，地图边线和障碍区域的几何形变明显减弱，有利于火源识别和路线判断。

附录 F 镜头畸变校正算法说明

由于实际图传摄像头存在广角畸变，画面边缘会出现弯曲，导致地图边线、障碍物轮廓和火源位置发生偏移。系统采用基于径向畸变模型的图像重映射方法，对每一帧进行实时校正。径向畸变可近似表示为：

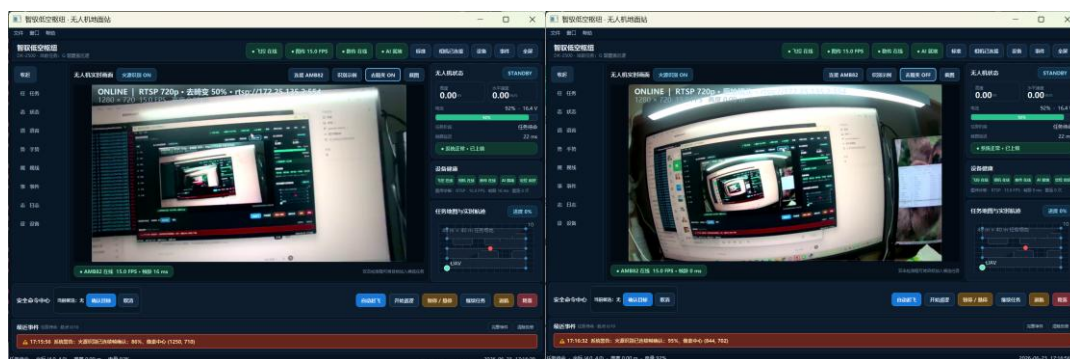
$$\begin{aligned} x_{\text{corrected}} &= x * (1 + k1*r^2 + k2*r^4) \\ y_{\text{corrected}} &= y * (1 + k1*r^2 + k2*r^4) \end{aligned}$$

其中：

- x、y 为归一化图像坐标；
- r 为像素点到图像中心的归一化距离；
- k1、k2 为畸变校正参数；
- 校正强度越大，边缘拉伸越明显。

程序中根据滑动条选择不同强度：

档位	适用场景
35	畸变较轻，画面边缘轻微弯曲
50	默认档位，兼顾边缘校正与画面稳定
65	畸变明显，地图边缘弯曲严重



上图中，左图为去畸变后，右图为无去畸变效果。

附录 G 火源识别算法说明

火源识别模块主要针对比赛场地中的红色模拟火源进行检测。算法不依赖大型深度学习模型，而是采用颜色空间分析、形态学滤波、连通域筛选和连续帧确认机制，具有计算量小、运行稳定、适合边缘设备部署的特点。

火源识别流程如下：



图 G 火源识别流程图

核心伪代码如下：

```
def detect_fire(frame):  
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)  
  
    mask_red_1 = cv2.inRange(hsv, lower_red_1, upper_red_1)  
    mask_red_2 = cv2.inRange(hsv, lower_red_2, upper_red_2)  
    mask = mask_red_1 | mask_red_2  
  
    mask = morphology_open(mask)  
    mask = morphology_close(mask)  
  
    contours = cv2.findContours(mask)  
  
    candidates = []  
    for c in contours:  
        area = cv2.contourArea(c)  
        if area < MIN_FIRE_AREA:  
            continue  
  
        x, y, w, h = cv2.boundingRect(c)  
        ratio = w / h  
  
        if 0.5 < ratio < 1.8:  
            candidates.append((x, y, w, h))  
  
    return candidates
```

为了避免单帧误检，系统采用连续帧确认：

```
if current_frame_detected_fire:  
    fire_confirm_count += 1  
else:  
    fire_confirm_count = max(0, fire_confirm_count - 1)
```

```
if fire_confirm_count >= CONFIRM_FRAME_COUNT:  
    fire_detected = True  
    publish_event("发现火源")
```

当火源被确认后，系统会执行以下动作：

1. 主画面绘制红色识别框；
2. 事件中心高亮显示火源告警；
3. 任务地图对应位置由蓝点变为红点；
4. 红点持续闪烁，提醒操作者；
5. 记录火源置信度、像素中心和地图位置。

附录 H 多模态语音识别模块

语音识别模块用于降低操作者在复杂环境下的操作负担。系统采用离线语音识别模型，将语音转写为文本后，再通过关键词匹配生成候选任务指令。

语音识别流程如下：

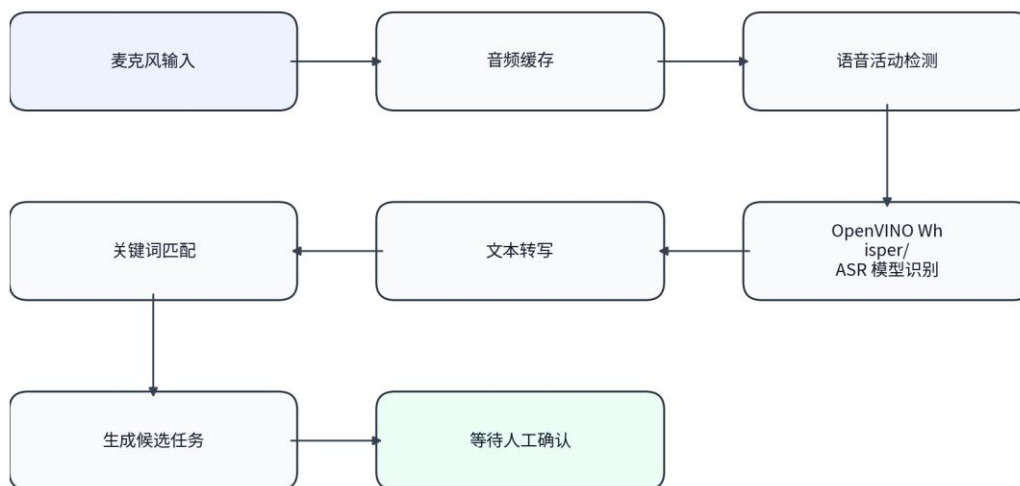


图 H 语音识别候选任务流程图

支持的典型语音指令包括：

语音内容	系统识别意图
开始巡逻	启动巡线任务
自动起飞	执行起飞流程
低空起飞	执行低空起飞流程
系统自检	执行无人机自检
返回起点	执行返航任务
暂停任务	进入悬停等待
继续任务	恢复任务执行
立即降落	执行降落流程
清除告警	清除当前告警状态

语音指令不会直接执行，而是显示为候选命令。例如识别到“开始巡逻”后，界面会生成“候选任务：开始巡逻”，操作者需要再次确认，系统才会向飞控发送任务指令。

附录 I 手势识别模块

手势识别模块用于在嘈杂环境或不便语音输入时进行任务选择。系统当前设计了三种基础手势，分别对应三类任务入口。

手势	含义	作用
张开手掌	Open Palm	选择任务类操作
握拳	Fist	选择暂停、悬停或取消类操作
竖大拇指	Thumb Up	选择确认、返航或完成类操作

手势识别基于手部关键点检测。程序首先定位手掌、指尖、指根等关键点，然后根据手指伸展状态判断当前手势。

手指伸展判断逻辑如下：

```
def is_finger_extended(tip, pip, wrist):  
    tip_dist = distance(tip, wrist)  
    pip_dist = distance(pip, wrist)  
    return tip_dist > pip_dist
```

张开手掌识别条件：拇指、食指、中指、无名指和小指均处于伸展状态时，系统判定为 Open Palm。

握拳识别条件：多数指尖靠近掌心，且伸展手指数小于或等于 1 时，系统判定为 Fist。

竖大拇指识别条件：拇指伸展、其他四指收拢，且拇指方向明显向上时，系统判定为 Thumb Up。

为提高稳定性，系统不会因为单帧识别结果立即触发任务，而是要求同一手势持续稳定一段时间。例如手势连续稳定 1 秒以上，才会被认为是有效输入。

附录 J 视线与头部姿态识别模块

视线识别模块采用“头部姿态为主、眼部方向为辅”的设计。由于普通摄像头对眼球细节捕捉能力有限，单纯依赖眼球位置容易受光照、眼镜反光、摄像头角度等因素影响。因此系统将头部偏转作为主要判断依据，用于实现左、中、右三个方向的选择。识别逻辑如下：



图 J 视线与头部姿态识别流程图

方向判断规则示例：

头部姿态	判定结果
$\text{yaw} < -\text{阈值}$	LEFT
$-\text{阈值} \leq \text{yaw} \leq \text{阈值}$	CENTER
$\text{yaw} > \text{阈值}$	RIGHT

视线模块与手势模块组合后，可形成“ 3×3 ”的任务选择方式。手势选择任务大类，视线选择该任务下的具体选项。例如：

手势	左视线	中视线	右视线
张开手掌	低空起飞	系统自检	开始巡线
握拳	暂停悬停	取消任务	继续任务
竖大拇指	立即返航	确认目标	执行降落

该设计将复杂按钮操作压缩为“一个手势 + 一个视线方向 + 二次确认”，降低了操作者在紧急情况下的操作负担。

附录 K 三卡片两阶段任务选择机制

为了避免手势或视线误识别造成危险动作，系统采用“三卡片两阶段”的交互方式。第一阶段只进行候选任务选择，第二阶段才进行任务确认。

第一阶段：任务选择。



图 K-1 三卡片第一阶段任务选择流程图

第二阶段：任务确认。

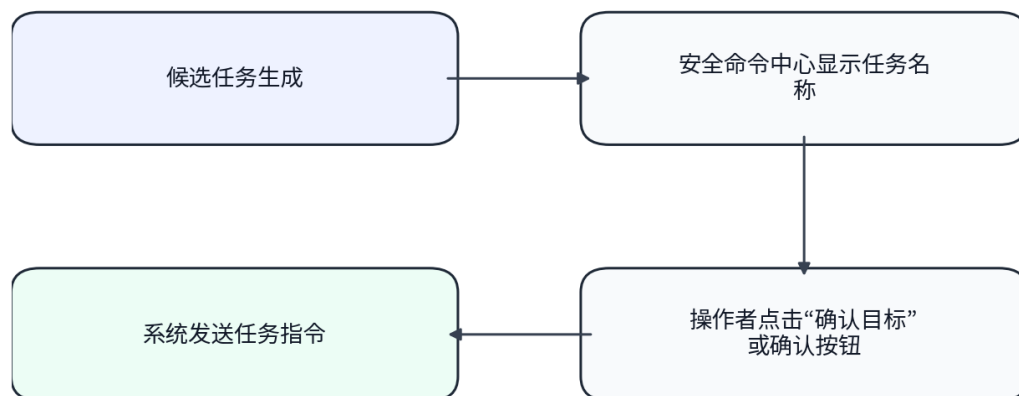


图 K-2 三卡片第二阶段任务确认流程图

该机制的优点是：

6. 手势不会直接控制无人机；
7. 视线只负责选择候选项；
8. 危险动作必须经过二次确认；
9. 任务流程可被事件日志完整记录；
10. 误识别时可以取消，不会立即触发飞控动作。

附录 L 任务状态机设计

地面站使用任务状态机管理无人机的任务流程。不同任务之间不能随意跳转，必须满足状态转换条件。

主要状态如下：

状态	含义
IDLE	空闲待命
PRECHECK	系统自检
WARMUP	起飞预热
TAKEOFF_LOW	低空起飞
TAKEOFF_HIGH	高空起飞
PATROL	巡线任务
FIRE_FOUND	发现火源
HOVER	悬停等待
RETURN_HOME	一键返航
LANDING	自动降落
EMERGENCY	异常处理

状态转换示例：

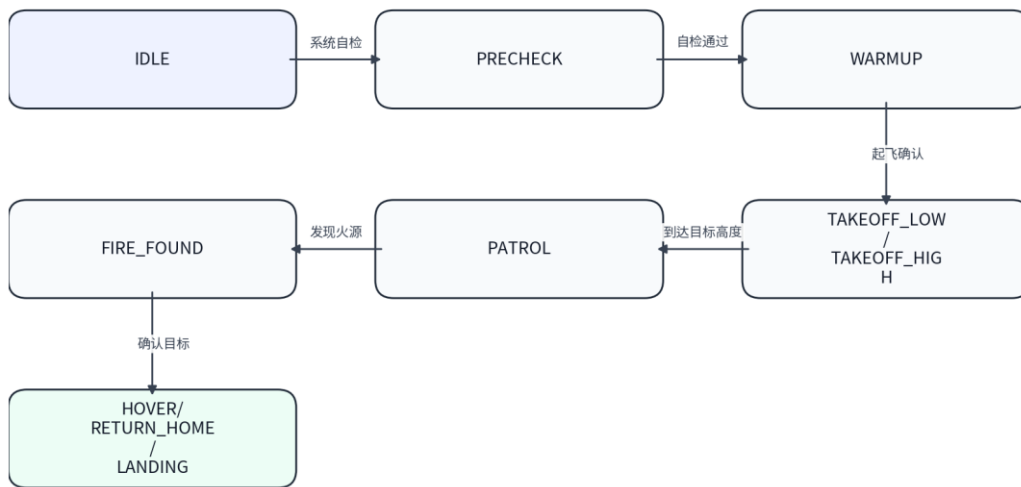


图 L-1 任务状态机转换流程图

状态机伪代码如下：

```

class MissionStateMachine:
    def __init__(self):
        self.state = "IDLE"

    def handle_event(self, event):
        if self.state == "IDLE":
            if event == "SELF_CHECK":
                self.state = "PRECHECK"

        elif self.state == "PRECHECK":
            if event == "CHECK_OK":
                self.state = "WARMUP"

        elif self.state == "WARMUP":

```

```
if event == "TAKEOFF_LOW":
    self.state = "TAKEOFF_LOW"

elif self.state == "TAKEOFF_LOW":
    if event == "ALTITUDE_REACHED":
        self.state = "PATROL"

elif self.state == "PATROL":
    if event == "FIRE_DETECTED":
        self.state = "FIRE_FOUND"

elif self.state == "FIRE_FOUND":
    if event == "RETURN_HOME":
        self.state = "RETURN_HOME"
```

状态机保证了任务流程的安全性。例如无人机未完成自检时，不能直接进入巡线任务；无人机处于降落状态时，不能再触发起飞任务。

附录 M 一键起飞、自检、返航任务说明

M.1 一键起飞

一键起飞任务分为预热、低空起飞和高空起飞三种形式。

类型	说明
预热	检查陀螺仪、电机、摄像头、数传连接是否正常
低空起飞	起飞至较低安全高度，用于近距离测试或室内演示
高空起飞	起飞至任务巡航高度，用于正式巡检任务

起飞流程如下：



图 M-1 一键起飞任务流程图

M.2 系统自检

自检任务主要检查以下内容：

项目	检查内容
陀螺仪 / IMU	角速度、姿态角、采样频率
电机 / ESC	电调连接、电机响应、PWM 输出
摄像头 / 图传	图传地址、分辨率、帧率、延迟
数传链路	串口连接、心跳包、数据延迟
电池	电压、电量、低电量告警

自检报告示例：

飞行器系统自检报告

陀螺仪 / IMU:

连接状态: 正常

驱动响应: OK

采样频率: 200 Hz

当前角度: Roll +0.1°, Pitch +0.1°, Yaw 11.2°

电机 / ESC:

连接状态: 4 路电调在线

驱动响应: PWM 回路正常

当前转速: 低速待命

摄像头 / 图传：
连接状态：AMB82 在线
当前地址：rtsp://172.25.135.2:554
画面参数：1280 × 720, 15 FPS, H.264

M.3 一键返航

返航任务用于在任务完成、发现火源或操作者主动触发时，使无人机返回起飞区。
返航流程如下：



图 M-2 一键返航任务流程图

附录 N 巡线任务与路径规划说明

巡线任务基于比赛场地地图进行规划。场地尺寸为 $48\text{ dm} \times 40\text{ dm}$ ，包含起飞区、障碍区域、巡检区域和模拟火源点。系统将场地抽象为二维坐标平面，并将无人机巡检路径表示为一系列航点。

航点结构如下：

```
@dataclass
class Waypoint:
    index: int
    x: float
    y: float
    altitude: float
    action: str = "PASS"
```

路径规划采用规则化航点策略。系统优先避开障碍区域，沿场地可通行区域生成巡检路线，并在关键区域设置火源检测点。

示例航点序列：

```
WP0: 起飞区
WP1: 左下巡检点
WP2: 左上巡检点
WP3: 中上巡检点
WP4: 右上巡检点
WP5: 右中巡检点
WP6: 中下巡检点
WP7: 返航点
```

巡线过程中，地面站根据当前航点更新地图蓝色标记。当火源识别成功后，对应检测点由蓝色变为红色，并持续闪烁，表示该点已经被识别为异常目标。

附录 O 地面站与飞控通信协议

地面站通过数传链路与飞控进行通信。通信内容包括任务指令、状态回传、心跳包、异常告警和任务确认信息。

通信数据分为两类：

类型	方向	内容
控制指令	地面站 → 飞控	起飞、巡线、返航、降落、悬停
状态回传	飞控 → 地面站	高度、电量、姿态、速度、任务阶段

任务指令示例：

```
{
  "type": "COMMAND",
  "command": "START_PATROL",
  "target_altitude": 1.8,
  "mission_id": "DK2500_PATROL_001",
  "timestamp": 1719300000
}
```

状态回传示例：

```
{
  "type": "STATUS",
  "mode": "AUTO",
  "altitude": 1.80,
  "velocity": 0.0,
  "battery": 92,
  "voltage": 16.4,
  "roll": -0.1,
  "pitch": 0.0,
  "yaw": 11.2,
  "current_waypoint": 2,
  "link_delay": 31
}
```

系统通过心跳包判断链路是否正常：

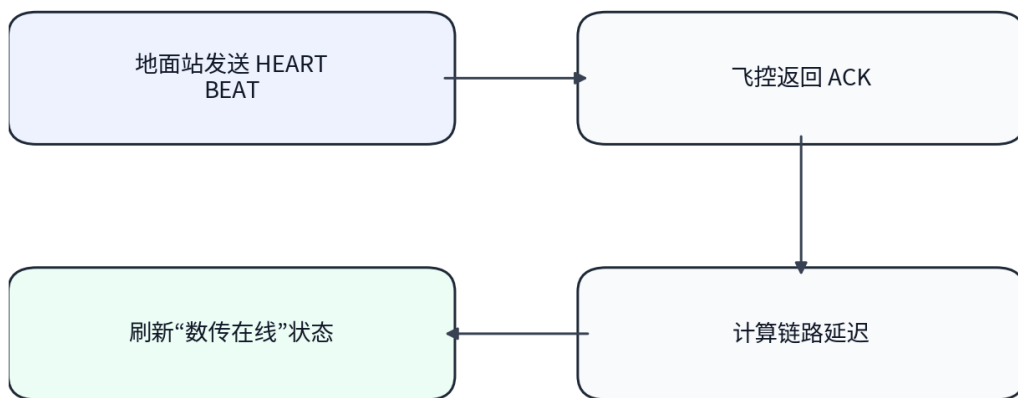


图 O-1 心跳包与链路状态判断流程图

若连续多次未收到心跳回复，地面站会提示“数传链路异常”，并禁止继续发送高风险任务。

附录 P MAVLink 协议栈说明

在正式飞控通信中，系统可兼容 MAVLink 协议。MAVLink 是无人机系统中常用的轻量级通信协议，适合串口、无线数传和低带宽链路。

MAVLink 协议栈可以分为四层：

层级	内容
应用层	WAYPOINT、COMMAND_LONG、MISSION_ITEM、HEARTBEAT
帧封装层	STX、LEN、MSGID、PAYLOAD、CHECKSUM
传输层	UART、Wi-Fi、nRF24L01
物理层	串口线、无线模块、开发板接口

常用消息包括：

消息	作用
HEARTBEAT	判断飞控是否在线
COMMAND_LONG	发送起飞、降落、返航等命令
ATTITUDE	获取姿态角
GLOBAL_POSITION_INT	获取位置和高度
MISSION_ITEM	发送航点任务
MISSION_ACK	确认任务接收状态

地面站在发送任务指令时，会将高层任务转换为飞控可理解的 MAVLink 命令。例如“开始巡线”会转换为多个航点任务，“一键返航”会转换为返航指令，“自动降落”会转换为降落指令。

附录 Q 安全命令与二次确认机制

无人机系统涉及起飞、降落、返航等高风险动作，因此本系统将所有危险动作划分为安全命令，并要求二次确认。
安全命令包括：

命令	风险等级	是否需要确认
自动起飞	高	是
开始巡线	中	是
暂停悬停	中	是
一键返航	高	是
自动降落	高	是
清除告警	中	是

候选命令流程如下：



图 Q-1 安全命令候选确认流程图

这种设计避免了识别算法直接控制无人机，提高了系统在复杂环境下的安全性。

附录 R 事件日志与告警机制

事件日志模块用于记录系统运行过程中的关键事件。记录内容包括事件时间、事件类型、来源模块、重要程度和处理状态。

事件格式如下：

```
@dataclass
class EventRecord:
    time: str
    level: str
    source: str
    message: str
    position: tuple | None = None
    confirmed: bool = False
```

事件类型包括：

类型	示例
系统事件	地面站启动、任务初始化
图传事件	摄像头连接成功、图传中断
识别事件	发现火源、火源置信度更新
任务事件	开始巡线、暂停任务、返航
告警事件	电量低、数传异常、识别失败

当识别到火源时，事件中心会变为红色告警状态，并显示最近火源识别记录：

```
发现火源
置信度：92%
地图位置：第二行第二列
像素中心：(420, 590)
处理状态：等待确认
```


附录 S UI 模块说明

地面站 UI 采用深色科技风格，整体布局分为顶部状态栏、左侧多模态交互区、中间主画面、右侧无人机状态区、底部安全命令中心和系统日志区。

区域	功能
顶部状态栏	显示飞控、图传、数传、AI 状态
左侧多模态交互区	语音、手势、视线输入状态
中间主画面	显示无人机图传、火源识别框、切换动画
右侧状态区	显示高度、电量、速度、姿态、任务地图
底部安全命令中心	显示候选任务和确认按钮
日志区	记录系统事件、告警和任务状态

UI 设计强调“主画面优先、状态常驻、安全命令可见”。摄像头画面作为主窗口，状态面板和安全按钮始终显示，避免比赛演示时因窗口缩放导致关键信息不可见。

附录 T 异常处理机制

系统针对常见异常设计了对应处理策略。

异常	检测方式	处理策略
图传中断	连续多帧读取失败	显示离线状态，尝试重连
火源误检	单帧识别异常	使用连续帧确认过滤
语音误识别	关键词置信度低	只生成候选命令，不直接执行
手势抖动	手势持续时间不足	不触发任务
视线偏移	头部姿态置信度低	保持上一稳定选项
数传异常	心跳超时	禁止发送高风险命令
电量过低	电压低于阈值	提示返航或降落
任务冲突	状态机不允许跳转	拒绝任务并记录日志

异常处理伪代码如下：

```
def safe_send_command(command):
    if not telemetry_online:
        log_warning("数传离线，禁止发送命令")
        return False

    if command in HIGH_RISK_COMMANDS and not user_confirmed:
        log_warning("高风险命令需要二次确认")
        return False

    if not mission_state_machine.can_execute(command):
        log_warning("当前状态不允许执行该命令")
        return False

    send_to_flight_controller(command)
    return True
```

附录 U 系统运行环境

地面站程序运行环境如下：

项目	配置
操作系统	Windows 10 / Windows 11
Python	Python 3.10 或以上
UI 框架	PyQt5 / PySide 系列
图像处理	OpenCV
手势识别	MediaPipe
语音识别	OpenVINO Whisper / ASR
图传设备	AMB82-mini
通信方式	串口 / 无线数传 / RTSP 图传
开发板	Intel DK-2500
飞控平台	F260-T432 无人机平台

运行步骤：

1. 安装 Python 环境
2. 安装依赖库
3. 连接 AMB82 图传模块
4. 启动地面站
5. 测试图传连接
6. 启动多模态识别
7. 执行自检、起飞、巡线、返航等任务

附录 V 关键参数表

参数	当前设置	说明
图传分辨率	1280 × 720	兼顾清晰度与延迟
图传帧率	15 FPS	适合比赛演示与识别
火源确认帧数	3 帧以上	防止单帧误检
手势稳定时间	约 1 秒	防止手势抖动
视线方向数	3 个	左、中、右
起飞高度	低空 / 高空可选	根据任务场景选择
巡线航点数	6 个左右	覆盖主要任务区域
电池告警阈值	可配置	低电量提示返航
镜头校正档位	35 / 50 / 65	适应不同畸变程度

附录 W 软件设计特点总结

本系统软件设计具有以下特点：

11. 多模态输入融合

支持语音、手势、视线、按钮等多种输入方式，降低操作者对传统遥控器的依赖。

12. 任务级命令封装

将复杂飞行动作封装为“自检、起飞、巡线、返航、降落”等任务级指令，降低使用门槛。

13. 安全二次确认

所有高风险动作均需要确认，避免误识别导致无人机误动作。

14. 轻量化火源识别

使用颜色空间和连续帧确认机制实现低计算量火源识别，适合边缘计算平台。

15. 图传与地图联动

火源识别结果能够同步显示在主画面、事件中心和任务地图中。

16. 任务状态机约束

不同任务之间按照合法状态转换执行，提高系统稳定性。

17. 模块化结构

图传、识别、通信、UI、日志等模块相互独立，便于后续替换和升级。

18. 面向比赛演示优化

UI 保留关键状态常驻显示，任务流程清晰，便于评委理解系统功能闭环。

附录 X 后续可扩展方向

在现有系统基础上，后续可以进一步扩展以下功能：

19. 接入更复杂的目标识别模型，例如 YOLO 系列火源检测模型；
20. 增加多无人机协同任务分配；
21. 引入更精确的室内定位方案，例如 UWB 或视觉定位；
22. 将任务日志导出为 CSV 或 PDF，便于赛后分析；
23. 增加真实飞行数据回放功能；
24. 支持地图编辑器，自定义障碍区和巡检路线；
25. 增加任务仿真模式，用于赛前训练；
26. 增加网络远程监控界面，实现多端查看无人机状态。